Arab Academy for Science, Technology, and Maritime Transport

College of Computing and Information Technology

Department of Computer Science

B.Sc. Final Year Project

DocMind

AI-Powered Academic Learning Assistant

Presented By:

Abdulrhman Mohamed Gomaa *Ahmed Mohamed Abo El Naga*

Mohamed Amgad Refat *Mohamed Ahmed Younes*

Amr Moustafa *Youssef Kamal*

Supervised By:

Dr. Ahmed Saied

June 2026

DECLARATION

We hereby certify that this material, which we now submit for assessment on the program of study leading to the award of Bachelor of Science in Computer Science, is entirely our own work. We have exercised reasonable care to ensure that the work is original, and does not, to the best of our knowledge, breach any law of copyright, and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of our work.

Student Name: Abdulrhman Mohamed Gomaa
Registration No.: 221007510
Signed: _____
Date: 29 June 2026

Student Name: Ahmed Mohamed Abo El Naga
Registration No.: 221006258
Signed: _____
Date: 29 June 2026

Student Name: Mohamed Amgad Refat
Registration No.: 221006487
Signed: _____
Date: 29 June 2026

Student Name: Mohamed Ahmed Younes
Registration No.: 222004133
Signed: _____
Date: 29 June 2026

Student Name: Amr Moustafa
Registration No.: 232004055
Signed: _____
Date: 29 June 2026

Student Name: Youssef Kamal
Registration No.: 221027780
Signed: _____
Date: 29 June 2026

ABSTRACT

DocMind is an AI (Artificial Intelligence)-powered academic learning assistant designed to enhance the learning experience of university students by enabling reliable, document-grounded conversational interactions. The system addresses common challenges faced by students when dealing with large volumes of academic materials by allowing them to query trusted content using natural language. Unlike general-purpose AI chat systems, DocMind employs Retrieval-Augmented Generation (RAG) so that answers are grounded in an explicit corpus: instructor-uploaded materials for subject-based chat, and student-uploaded Portable Document Format (PDF) files for document-based chat.

The implemented system provides role-based access control (RBAC) for students, instructors, and administrators enforced in the Application Programming Interface (API) and web client, JSON Web Token (JWT)-based authentication with configurable token lifetime, administrator-controlled student-access gating, and validated document uploads with per-conversation limits. The backend is built on FastAPI with a PostgreSQL relational database, while semantic retrieval is powered by a pluggable vector store supporting both pgvector and Qdrant. The React-based web frontend delivers a responsive Single-Page Application (SPA) accessible across desktop and mobile browsers, complemented by a native Flutter mobile application for iOS and Android that consumes the same API. This document presents a comprehensive overview of the system, covering the literature review, system analysis and design, and implementation details, demonstrating the feasibility of integrating trustworthy AI-driven assistance into formal academic environments.

الملخص

DocMind هو مساعد تعليمي أكاديمي مدعوم بالذكاء الاصطناعي، صُمِّم لتعزيز تجربة التعلم لدى طلاب الجامعات من خلال توفير تفاعلات محادثية موثوقة ومرتكزة على المستندات. يعالج النظام التحديات الشائعة التي يواجهها الطلاب عند التعامل مع كميات كبيرة من المواد الأكاديمية، إذ يتيح لهم الاستعلام عن المحتوى الموثوق باستخدام اللغة الطبيعية.

على خلاف أنظمة الدردشة بالذكاء الاصطناعي ذات الأغراض العامة، يعتمد دوك مايند على تقنية التوليد المعزز بالاسترجاع (RAG) بحيث ترتكز الإجابات أساساً على مجموعة محتوى محددة: المواد التي يرفعها أعضاء هيئة التدريس لمحادثات المساعد الافتراضي (الدردشة بالمادة)، والملفات التي يرفعها الطلاب أنفسهم في محادثات المستندات. يدعم النظام وضعَي تفاعل رئيسيَّين: الدردشة المرتكزة على المستندات، والدردشة المرتكزة على مواد المادة الدراسية.

يوفر النظام المُنفَّذ تحكماً في الوصول قائماً على الأدوار للطلاب والمدرسين والمسؤولين، ومصادقة مبنية على رموز JWT مع مدة صلاحية قابلة للضبط، وبوابة للتحكم في وصول الطلاب يديرها المسؤول، فضلاً عن رفع مستندات محقَّق منها مع حدود لكل محادثة. تم بناء الواجهة الخلفية باستخدام FastAPI مع قاعدة بيانات علائقية PostgreSQL، فيما يعتمد الاسترجاع الدلالي على مخزن متجهات قابل للتوصيل يدعم كلاً من pgvector وQdrant. تقدم الواجهة الأمامية المبنية بـReact تطبيقاً من صفحة واحدة يعمل على أجهزة سطح المكتب والهاتف المحمول.

يقدم هذا المستند نظرة شاملة على النظام تغطي مراجعة الأدبيات، وتحليل النظام وتصميمه، وتفاصيل التنفيذ، مما يعكس التطبيق الفعلي للمستودع ويُثبت جدوى دمج المساعدة الموثوقة المدعومة بالذكاء الاصطناعي في البيئات الأكاديمية الرسمية.

DEDICATION

To our families, whose patience and support made this journey possible.

To our supervisor, Dr. Ahmed Saied, for his invaluable guidance.

And to every student who deserves a smarter, more accessible way to learn.

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to Dr. Ahmed Saied for his continuous guidance, insightful feedback, and unwavering support throughout the development of this project. His expertise and encouragement were instrumental in shaping both the academic and technical aspects of DocMind.

We also extend our thanks to the faculty and staff of the College of Computing and Information Technology at the Arab Academy for Science, Technology, and Maritime Transport, for providing an environment conducive to research and innovation.

Special appreciation goes to our families and friends for their patience, encouragement, and moral support during the many hours invested in this project.

Finally, we acknowledge the open-source communities behind FastAPI, React, PostgreSQL, LangChain, pgvector, and Qdrant, whose contributions to the software ecosystem made the technical realization of DocMind possible.

CONTENTS

Declaration 1

Abstract 2

الملخص (Arabic Abstract) 3

Dedication 4

Acknowledgments 5

List of Tables 12

List of Figures 13

List of Acronyms / Abbreviations 14

1 Introduction 15

1.1 Background 15

1.2 Problem Statement 16

1.3 Motivation 16

1.4 Objectives 17

1.5 Scope 17

1.6 Organization of the Document 18

2 Literature Review **19**

2.1 Research Papers 19

2.1.1 Paper 1: Comparing Retrieval-Augmentation and Parameter Efficient Fine-Tuning for Privacy-Preserving Personalization of Large Language Models [1] 20

2.1.2 Paper 2: A Comparison of Independent and Joint Fine-Tuning Strategies for Retrieval-Augmented Generation [2] 24

2.1.3 Paper 3: Fine-Tuning or Fine-Failing? Debunking Performance Myths in Large Language Models [3] 28

2.1.4 Paper 4: Enhancing Retrieval-Augmented Generation: A Study of Best Practices [4] 31

2.2 Market Analysis 35

2.3 Related Projects and Similar Systems 36

2.3.1 NotebookLM 36

2.3.2 Humata 40

2.3.3 Mindgrasp 42

2.3.4 AskYourPDF 44

2.4 Market Survey Summary 46

3 Proposed Work: System Analysis and Design **47**

3.1 System Overview 47

3.2 System Operating Environment 48

3.3 User Characteristics 48

3.4 System Functions 49

3.5 System Constraints 50

3.6 Assumptions and Dependencies 51
3.7 SWOT Analysis 52
3.7.1 Strengths 52
3.7.2 Weaknesses 52
3.7.3 Opportunities 52
3.7.4 Threats 53
3.8 Functional Requirements 54
3.8.1 User Roles and Characteristics 54
3.8.2 Authentication and Authorization Requirements 55
3.8.3 Document Management Requirements 56
3.8.4 Chat and Interaction Requirements 57
3.8.5 Session Management Requirements 58
3.8.6 Administrative and Monitoring Requirements 59
3.9 Nonfunctional Requirements 60
3.9.1 Performance Requirements 60
3.9.2 Security Requirements 60
3.9.3 Usability Requirements 60
3.9.4 Reliability and Availability Requirements 61
3.9.5 Scalability Requirements 61
3.10 Constraints and Limitations 61

3.11 System Design 62
3.11.1 Design Goals and Principles 62
3.11.2 Architectural Overview 62
3.11.3 Presentation Layer Design 63
3.11.4 Application Layer Design 64
3.11.5 AI Services Layer Design 65
3.11.6 Data Storage Layer Design 66
3.11.7 System Block Diagram 67
3.11.8 Use Case Diagram 68
3.11.9 Activity Diagram 69
3.11.10 Sequence Diagrams 70
3.11.11 Database Design and ERD 72
3.11.12 Security Design 74
3.11.13 Error Handling and Fault Tolerance 74
3.11.14 Design Validation 74

4 Implementation and Testing **75**

4.1 Technology Stack 75

4.2 Backend Implementation 77

4.2.1 Project Structure 77

4.2.2 Authentication and Authorization 77

4.2.3 Document Ingestion Pipeline 78

4.2.4 RAG Query Pipeline 79

4.2.5 Administrative APIs 79

4.3 Frontend Implementation 80

4.3.1 Application Structure 80

4.3.2 State Management 80

4.4 Student Interface 81

4.4.1 Document-Based Chat Interface 81

4.4.2 Subject-Based Chat Interface 82

4.5 Instructor Interface 83

4.5.1 Content Management Interface 83

4.6 Testing 84

4.6.1 Testing Strategy 84

4.6.2 Test Environment 84

4.6.3 Test Cases 85

4.6.4 Known Limitations and Remaining Gaps 86

4.7 Prototype Evaluation 86

5 Conclusions and Future Work **87**

5.1 Conclusions 87

5.2 Future Work 89

5.2.1 Single Sign-On Integration 89

5.2.2 Multi-Language Support 89

5.2.3 Automated Quiz and Assessment Generation 89

5.2.4 Enhanced Retrieval Strategies 90

5.2.5 Analytics and Learning Insights 90

5.2.6 Production Hardening 90

5.2.7 Agentic Retrieval Expansion 90

References **91**

LIST OF TABLES

2.1 Evaluation Results on HotPotQA and PopQA 26

2.2 Ablation Study Results on TruthfulQA and MMLU 33

2.3 Market Survey Comparison of Similar Systems 46

3.1 Authentication and Authorization Requirements 55

3.2 Document Management Requirements 56

3.3 Chat and Interaction Requirements 57

3.4 Session Management Requirements 58

3.5 Administrative and Monitoring Requirements 59

4.1 Representative System Test Cases 85

LIST OF FIGURES

2.1 Comparison of Personalization Techniques Across LaMP Datasets 22

2.2 Different fine-tuning techniques for RAG pipelines: (a) fine-tune embedding model using context labels; (b) freeze generator while fine-tuning the embedding model; (c) fine-tune embedding model jointly; (d) two-phase fine-tuning. . 25

2.3 Model evaluation results comparing Exact Match (EM), F1, Recall@5, and Time (h) across fine-tuning strategies. 26

2.4 Accuracy of fine-tuned LLaMA-2 models across BioASQ, Natural Questions (NQ), and QASPER datasets. 29

2.5 Accuracy of fine-tuned Mistral models across BioASQ, Natural Questions (NQ), and QASPER datasets. 30

3.1 DocMind System Block Diagram. 67

3.2 DocMind Use Case Diagram. 68

3.3 DocMind Full System Activity Diagram (Student POV). 69

3.4 Chat with Document (RAG) – Upload and Vector Check Sequence Diagram. . 70

3.5 Subject Chat (Instructor Material and RAG) – Sequence Diagram. 71

3.6 DocMind Entity-Relationship Diagram (ERD). 73

4.1 Student Interface – Document-Based Chat. 81

4.2 Student Interface – Subject-Based Chat (Chat with Tutors). 82

4.3 Instructor Interface – Content Management Dashboard. 83

LIST OF ACRONYMS / ABBREVIATIONS

Acronym	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CLI	Command-Line Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
ERD	Entity-Relationship Diagram
GDPR	General Data Protection Regulation
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICL	In-Context Learning
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LMS	Learning Management System
NLP	Natural Language Processing
PDF	Portable Document Format
PEFT	Parameter-Efficient Fine-Tuning
PostgreSQL	Post Ingres Structured Query Language
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
SRS	Software Requirements Specification
SSO	Single Sign-On
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The rapid advancement of information technology has significantly transformed modern educational environments. Universities and academic institutions increasingly rely on digital platforms to distribute learning materials, manage coursework, and facilitate communication between instructors and students. Lecture notes, presentations, assignments, and supplementary references are now commonly delivered through online Learning Management Systems (LMS), shared repositories, and institutional portals.

While this digital transformation has improved accessibility and flexibility, it has also introduced new challenges. Students are often required to process large volumes of academic material within limited timeframes. Navigating these materials efficiently, identifying relevant information, and fully understanding complex concepts has become increasingly difficult. Traditional learning methods, such as manual document reading or keyword-based searching, are often insufficient to support effective comprehension in such data-heavy environments.

At the same time, recent advancements in Artificial Intelligence (AI), particularly in Natural Language Processing (NLP), have enabled the development of conversational systems capable of interacting with users in human-like ways. These systems have demonstrated potential in education by providing personalized assistance, answering questions, and supporting self-directed learning in [6]. However, the integration of such technologies into formal academic environments requires careful consideration to ensure accuracy, reliability, and alignment with educational objectives.

1.2 PROBLEM STATEMENT

Despite the availability of extensive academic resources, students frequently encounter difficulties in extracting meaningful knowledge from them. Large documents such as lecture slides and textbooks often contain dense information, making it challenging for students to locate specific answers or clarify particular concepts efficiently. As a result, students may resort to external sources that are not aligned with course content, or may receive incomplete or misleading explanations.

Generic AI chat systems have gained popularity as tools for answering questions and providing explanations. However, these systems are typically trained on broad datasets and are not restricted to verified academic sources. Consequently, they may generate responses that are factually incorrect, inconsistent with course materials, or entirely fabricated — a phenomenon commonly referred to as AI hallucination in [7]. Such behavior poses significant risks in academic contexts, where accuracy and reliability are essential.

Furthermore, instructors often face repetitive questions from students regarding course materials, which can increase workload and reduce time available for higher-level academic engagement. There is a clear need for a system that can provide consistent, accurate, and course-aligned assistance while reducing reliance on unreliable external tools.

1.3 MOTIVATION

The motivation behind the DocMind project arises from the need to bridge the gap between traditional academic content and intelligent learning assistance. The project aims to leverage modern AI techniques while addressing the limitations and risks associated with unrestricted language models in education.

DocMind is motivated by the concept of grounded intelligence, where AI-generated responses are explicitly tied to trusted and verified sources. By restricting the system's knowledge base to instructor-approved documents and uploaded academic materials, DocMind seeks to ensure that all responses remain aligned with course objectives and institutional standards.

Another key motivation is to support self-directed learning. Students often learn at different paces and require varying levels of explanation. An interactive academic assistant allows students to ask questions freely, revisit concepts multiple times, and explore topics in greater depth without time constraints. This flexibility promotes independent learning while maintaining academic integrity.

1.4 OBJECTIVES

The primary objective of the DocMind project is to design and implement an AI-powered academic learning assistant that provides accurate, reliable, and context-aware support for university students. To achieve this goal, the project defines several specific objectives:

- To enable natural language interaction with academic documents and course materials.
- To ensure that all generated responses are grounded in verified and instructor-approved sources.
- To reduce the occurrence of AI hallucinations and misinformation in educational contexts.
- To support instructors by minimizing repetitive clarification requests.
- To provide a scalable and maintainable system architecture suitable for institutional deployment.
- To promote ethical and responsible use of artificial intelligence in education.

These objectives guide both the functional requirements and the design decisions of the system, ensuring alignment between user needs and technical implementation.

1.5 SCOPE

The scope of the DocMind project includes the design and development of an academic assistant integrated within a university environment. The system supports two primary modes of interaction: document-based chat, where students interact with uploaded documents, and subject-based chat, which relies on instructor-managed course materials.

The project focuses on system architecture, user interaction design, database design, and AI integration using Retrieval-Augmented Generation (RAG) techniques as described in [5]. The system is designed to operate within defined constraints, such as authenticated session expiry via JSON Web Token (JWT), per-role authorization, per-conversation upload limits and file-type validation, and administrator policies including optional disabling of student access, to ensure relevance and reliability.

The project does not aim to replace instructors, tutors, or formal assessment methods. Instead, DocMind functions as a supplementary learning tool that enhances understanding and supports academic engagement. Features such as grading automation or autonomous decision-making are explicitly outside the scope of this project.

1.6 ORGANIZATION OF THE DOCUMENT

This document is organized into five chapters to provide a comprehensive and structured presentation of the DocMind system.

1. **Chapter 1 (Introduction)** introduces the project background, problem statement, motivation, objectives, and overall scope of the system.
2. **Chapter 2 (Literature Review)** presents a detailed review of relevant research papers, market analysis, related projects, and similar existing systems, highlighting existing approaches and their implications for the proposed solution.
3. **Chapter 3 (Proposed Work — System Analysis and Design)** describes the overall behavior of the DocMind system, including functional and nonfunctional requirements, system architecture, AI pipeline design, database design, and Unified Modeling Language (UML) diagrams.
4. **Chapter 4 (Implementation and Testing)** documents the repository-backed implementation, interface walkthroughs for all user roles, prototype evaluation, and testing considerations.
5. **Chapter 5 (Conclusions and Future Work)** presents conclusions validating the project and outlines planned enhancements and directions for future development.

CHAPTER 2

LITERATURE REVIEW

This chapter presents a review of relevant academic research related to artificial intelligence in education, conversational learning systems, and document-grounded AI models. It also provides a market analysis and survey of similar systems currently available. The reviewed papers provide foundational insights that inform the design and implementation of the DocMind system. Rather than merely summarizing prior work, this chapter discusses the implications of each study for the DocMind approach, identifying strengths, limitations, and design decisions that arise from the literature.

2.1 RESEARCH PAPERS

This section reviews four research papers that examine fine-tuning and Retrieval-Augmented Generation (RAG), highlighting their performance, limitations, and suitability for the DocMind application.

2.1.1 Paper 1: Comparing Retrieval-Augmentation and Parameter Efficient Fine-Tuning for Privacy-Preserving Personalization of Large Language Models [1]

Authors: Hamed Zamani, Alireza Salemi
Published at: 26 Jun 2025, University of Massachusetts

Overview

This paper compares retrieval-augmented generation (RAG) and parameter-efficient fine-tuning (PEFT) for privacy-preserving personalization of large language models (LLMs). Results show that RAG outperforms PEFT when user data is limited, while PEFT improves with larger user profiles. Combining RAG and PEFT achieves the best personalization performance without violating user privacy.

The Dataset

LaMP datasets with different personalized fields:

- LaMP-1 (Citation)
- LaMP-2 (Movie Tagging)
- LaMP-3 (Product Rating)
- LaMP-7 (Tweet Paraphrasing)

Methodology

PEFT-RAG for Personalizing LLMs by Two Approaches:

1. **Personalizing the Input Prompt**: This approach modifies the input prompt given to the model to encourage it to generate responses that are more tailored to the specific user's needs.
2. **Changing the Model Parameters**: This approach involves training the LLM with user-specific data, allowing the model to learn the user's preferences.

This approach combines Parameter-Efficient Fine-Tuning (PEFT) and Retrieval-Augmented Generation (RAG) to enhance the personalization of large language models by:

PEFT: Fine-tunes a small set of additional parameters, reducing computational and memory overhead.

RAG: Mitigates the limitations of PEFT by dynamically retrieving user-specific information during inference.

Comparison of Different Techniques

The following figure compares the methods described — RAG, PEFT, and PEFT-RAG. As shown in Figure 2.1, the combination of both techniques consistently yields the highest personalization scores across the LaMP benchmark tasks.

Dataset	Metric	No Personalization	PEFT Personalization				RAG Personalization				PEFT-RAG Personalization			
			$r = 8$	$r = 16$	$r = 32$	$r = 64$	BM25	Recency	Contriever	RSPG	$r = 8$	$r = 16$	$r = 32$	$r = 64$
LaMP-1: Personalized Citation Identification	Accuracy ↑	0.502	0.502	0.502	0.504	0.506	0.626	0.622	0.636	**0.672**	0.670	0.668	0.671	0.671
LaMP-2: Personalized Movie Tagging	Accuracy ↑	0.359	0.360	0.360	0.360	0.359	0.387	0.377	0.396	0.430	0.430	**0.431**	0.430	0.430
	F1 ↑	0.276	0.278	0.278	0.278	0.277	0.306	0.295	0.304	0.339	0.341	**0.342**	0.341	0.341
LaMP-3: Personalized Product Rating	MAE ↓	0.308	0.308	0.307	0.306	0.301	0.298	0.296	0.299	0.264	0.264	0.265	0.264	**0.259**
	RMSE ↓	0.611	0.607	0.607	0.602	0.600	0.611	0.605	0.616	0.568	0.568	0.570	0.564	**0.562**
LaMP-4: Personalized News Headline Generation	ROUGE-1 ↑	0.176	0.178	0.177	0.178	0.178	0.186	0.189	0.183	0.203	0.203	**0.204**	**0.204**	0.203
	ROUGE-L ↑	0.160	0.162	0.162	0.163	0.163	0.171	0.173	0.169	0.186	0.186	0.186	**0.187**	0.186
LaMP-5: Personalized Scholarly Title Generation	ROUGE-1 ↑	0.478	0.478	0.478	0.477	0.478	0.477	0.475	**0.483**	0.480	0.481	0.480	0.480	0.479
	ROUGE-L ↑	0.428	0.429	0.429	0.428	0.428	0.427	0.426	**0.433**	0.429	0.431	0.431	0.431	0.431
LaMP-6: Personalized Email Subject Generation	ROUGE-1 ↑	0.335	0.342	0.342	0.341	0.343	0.412	0.403	0.401	0.433	0.436	0.436	0.436	**0.437**
	ROUGE-L ↑	0.319	0.325	0.326	0.325	0.326	0.398	0.389	0.386	0.418	**0.422**	**0.422**	**0.422**	0.421
LaMP-7: Personalized Tweet Paraphrasing	ROUGE-1 ↑	0.449	0.449	0.449	0.449	0.449	0.446	0.444	0.440	**0.461**	0.460	0.460	0.460	0.460
	ROUGE-L ↑	0.396	0.397	0.397	0.396	0.396	0.394	0.393	0.390	**0.409**	**0.409**	**0.409**	0.408	**0.409**

Figure 2.1: Comparison of Personalization Techniques Across LaMP Datasets

Main Findings

The combination of PEFT-trained personalized LLMs and RAG retrieval models resulted in:

a- 15.98% improvement over the non-personalized LLM.

b- Outperforming standard RAG in 4 out of 7 tasks, showing that PEFT enhances personalization by fine-tuning the model alongside real-time retrieval.

Drawbacks

1. RAG suffers from retrieval latency, particularly as the profile size increases.
2. PEFT is constrained by adapter loading overhead, which can impact real-time applications requiring frequent updates.

Implications for DocMind

The findings from this paper reinforce the decision to use RAG as the primary retrieval strategy in DocMind. Given that DocMind operates on institutional document corpora with no per-student parametric fine-tuning, the RAG-only approach remains practical, cost-effective, and privacy-preserving. Fine-tuning could be explored in future work once sufficient interaction data is available.

2.1.2 Paper 2: A Comparison of Independent and Joint Fine-Tuning Strategies for Retrieval-Augmented Generation [2]

Authors: Neal Lawton, Alfy Samuel, Anoop Kumar, Daben Liu
Published at: 17 Oct 2025, Cornell University

Overview

This paper compares independent, joint, and two-phase fine-tuning strategies for retrieval-augmented generation (RAG) systems. Experiments show that all strategies achieve similar answer quality improvements, but differ greatly in computational cost. The optimal strategy depends on whether context labels are available and whether extensive learning-rate tuning is required.

The Dataset

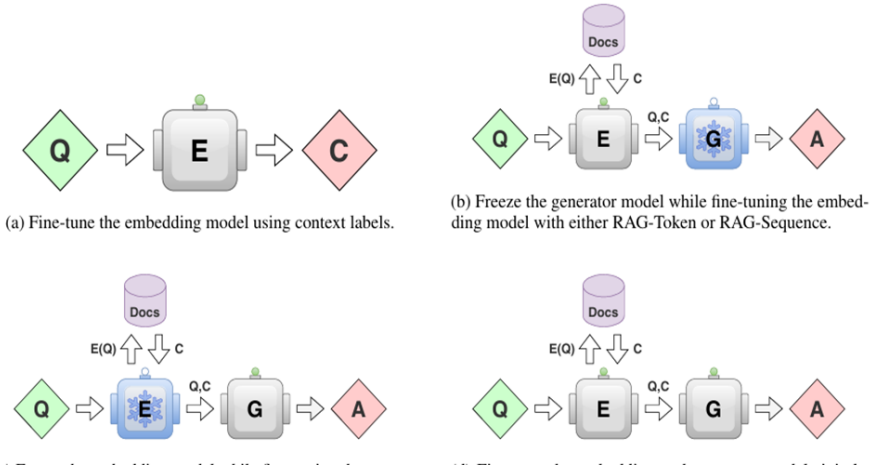
1. **HotPotQA** dataset including 113k Questions and Answers with distractors to make the reasoning harder and test the fine-tuning performance (general).
2. **PopQA** dataset: a smaller dataset containing 30k question-and-answer pairs including distractors (more specific).

Methodology

The paper evaluates five techniques for applying RAG:

1. No fine-tuning.
2. Fine-tuning the embedded vector in the RAG pipeline.
3. Fine-tuning the LLM Generator (highly recommended).
4. Fine-tuning both the RAG pipeline and the LLM Generator (joint fine-tuning).
5. Two-phase fine-tuning (fine-tuning the embedded vector alone, followed by fine-tuning the generator alone, then balancing between them).

Diagram of Different Techniques:



(a) Fine-tune the embedding model using context labels.

(b) Freeze the generator model while fine-tuning the embedding model with either RAG-Token or RAG-Sequence.

Figure 2.2: Different fine-tuning techniques for RAG pipelines: (a) fine-tune embedding model using context labels; (b) freeze generator while fine-tuning the embedding model; (c) fine-tune embedding model jointly; (d) two-phase fine-tuning.

Models Evaluation

Table 2.1 and Figure 2.3 present the evaluation results for each fine-tuning strategy across both datasets.

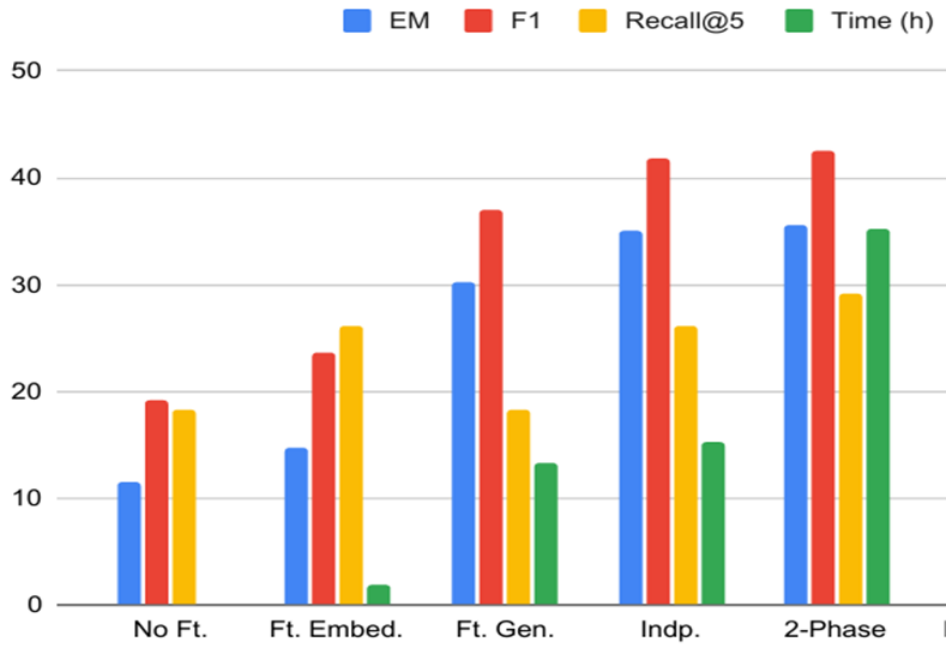


Figure 2.3: Model evaluation results comparing Exact Match (EM), F1, Recall@5, and Time (h) across fine-tuning strategies.

Table 2.1: Evaluation Results on HotPotQA and PopQA

Method	HotPotQA				PopQA			
	EM	F1	Recall@5	Time (h)	EM	F1	Recall@5	Time (h)
No Ft.	10.3	19.8	19.1	0.0	12.6	18.6	17.4	0
Ft. Embed.	11.1	20.8	21.4	3.5	18.2	26.6	30.8	0.4
Ft. Gen.	28.4	39.4	19.1	23.8	32.1	34.7	17.4	2.9
Indp.	29.3	40.2	21.4	27.4	40.6	43.2	30.8	3.2
2-Phase	30.0	41.3	25.1	61.0	41.0	43.7	33.3	9.4

Conclusion

Fine-tuning the generator only is the most realistic option for DocMind as it is easier to implement than joint fine-tuning and two-phase fine-tuning. It also yields acceptable results in large datasets, making it suitable for this use case. Finally, it is comparatively affordable compared to more complex fine-tuning techniques.

Implications for DocMind

This paper confirms that generator fine-tuning offers a practical middle ground between raw RAG and computationally expensive joint strategies. For DocMind, where inference budget and operational simplicity are priorities, generator-only fine-tuning represents a viable future enhancement that does not require restructuring the retrieval pipeline.

2.1.3 Paper 3: Fine-Tuning or Fine-Failing? Debunking Performance Myths in Large Language Models [3]

Authors: Scott Barnett, Zac Brannelly, Stefanus, Sheng Wong
Published at: 17 Jun 2025, Deakin University

Overview

This paper investigates whether fine-tuning large language models improves performance when used inside RAG pipelines. Experiments across multiple datasets show that fine-tuned models often perform worse than baseline models in both accuracy and completeness. The findings challenge the assumption that fine-tuning always helps, emphasizing the need for careful validation in domain-specific RAG settings.

The Dataset

The researchers tested their approach on three publicly available question-and-answer datasets, each from a different domain:

BioASQ:

- Biomedical question-and-answer dataset
- 15,000 documents
- 1,000 expert-created question-and-answer pairs for fine-tuning

Natural Questions (NQ):

A large-scale open-ended question-and-answer dataset containing real Google search queries with answers linked to Wikipedia pages.

QASPER:

- Dataset from NLP research papers
- 5,049 questions from 1,585 scientific papers

Methodology

1. **Models**: LLaMA-2, Mistral, and GPT-4
2. **Fine-Tuning**: Fine-tuned LLaMA-2 and Mistral on 200 samples, 500 samples, and 1,000 samples using LoRA / QLoRA for efficient fine-tuning. Hardware: A100/H100 GPUs.
3. **RAG**: For each question, the top 5 similar chunks were retrieved from the dataset, concatenated into one context block, and passed with the question to the model.

Comparison

Figure 2.4 and Figure 2.5 show the accuracy of fine-tuned models across all three datasets, demonstrating that fine-tuning consistently failed to improve results over the baseline.

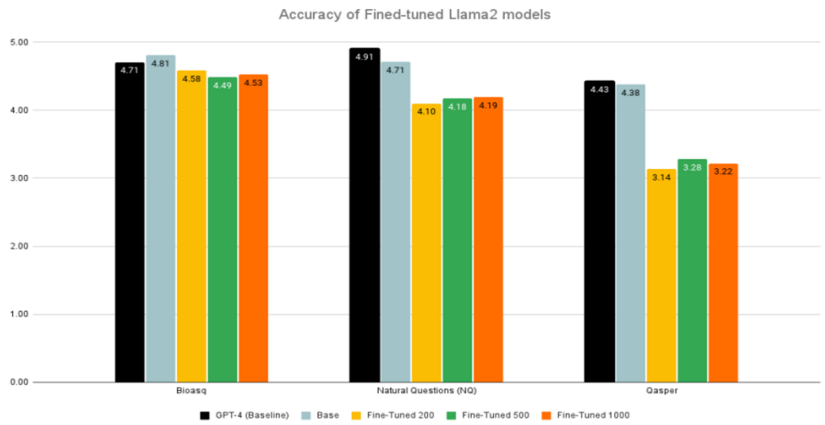


Figure 2.4: Accuracy of fine-tuned LLaMA-2 models across BioASQ, Natural Questions (NQ), and QASPER datasets.

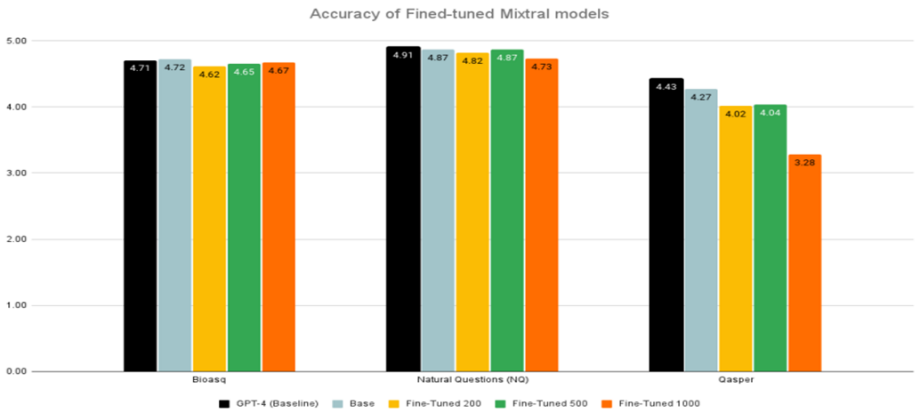


Figure 2.5: Accuracy of fine-tuned Mistral models across BioASQ, Natural Questions (NQ), and QASPER datasets.

1. Fine-tuning made the models perform worse.
2. The baseline (original) LLMs gave more accurate and complete answers than the fine-tuned versions.
3. Increasing dataset size from 200 to 500 to 1000 samples did not improve results, and in some cases made them worse.
4. Limitation of hardware resources negatively affects the fine-tuning process.

Implications for DocMind

The results in [3] directly validate the decision to avoid fine-tuning in the DocMind prototype. In specialized, document-grounded educational settings, the cost and risk of fine-tuning outweigh potential gains. Retrieval quality and prompt engineering are more reliable levers for improving answer quality, and these are the strategies adopted in the DocMind pipeline.

2.1.4 Paper 4: Enhancing Retrieval-Augmented Generation: A Study of Best Practices [4]

Authors: Siran Li, Linus Stenzel, Carsten Eickhoff, Seyed Ali
Published at: 13 Jan 2025, University of Tübingen

Overview

This paper systematically studies best practices for designing high-performance Retrieval-Augmented Generation (RAG) systems through extensive ablation experiments. Results show that Contrastive In-Context Learning (ICL) RAG and Focus Mode RAG consistently outperform other variants, especially on knowledge-intensive tasks such as the Massive Multitask Language Understanding (MMLU) benchmark. The study concludes that retrieval quality, prompt design, and targeted context selection matter more than larger knowledge bases or aggressive retrieval strategies.

The Dataset

TruthfulQA:

- 817 questions across 38 categories
- Tests commonsense knowledge and truthfulness
- Includes correct and incorrect answers for evaluation

MMLU (Massive Multitask Language Understanding):

- Educational and professional domains, 57 subjects
- Multiple-choice questions (1824 samples used for evaluation)
- Requires precise and specialized knowledge

Methodology

Query Expansion Module:

- Uses Flan-T5 to expand queries for more relevant retrieval.

Retrieval Module:

- Uses FAISS as described in [15] with Sentence Transformers for semantic search.
- Documents split into chunks; Focus Mode retrieves only the most relevant sentences.

Text Generation Model (LLM):

- Mistral models (7B and 45B) generate responses based on retrieved context.

Comparison

Table 2.2 summarizes the ablation study results across TruthfulQA and MMLU, comparing all RAG configurations evaluated in [4].

Table 2.2: Ablation Study Results on TruthfulQA and MMLU

		TruthfulQA					MMLU				
		R1	R2	RL	ECS	Mauve	R1	R2	RL	ECS	Mauve
LLM Size	Instruct7B	26.81	13.26	23.86	56.44	72.92	10.42	1.90	8.91	29.41	40.51
	Instruct45B	29.07	14.95	25.64	58.63	81.62	11.06	2.05	9.37	30.82	38.24
Prompt Design	HelpV1	26.81	13.26	23.86	56.44	72.92	10.42	1.90	8.91	29.41	40.51
	HelpV2	27.00	13.88	23.93	57.33	75.38	10.21	1.80	8.77	29.45	36.20
	HelpV3	26.30	13.01	23.16	56.54	79.20	10.40	1.97	9.00	29.39	34.50
	AdversV1	10.06	1.60	8.60	19.78	2.55	6.58	0.72	5.75	14.04	4.05
	AdversV2	8.39	2.14	7.48	16.30	0.93	4.24	0.54	3.84	12.33	0.76
Doc Size	2DocS	27.41	13.71	24.27	57.52	78.53	10.43	1.92	8.88	29.44	38.22
	2DocM	26.81	13.26	23.86	56.44	72.92	10.42	1.90	8.91	29.41	40.51
	2DocL	26.96	13.78	23.92	57.00	82.02	10.41	1.88	8.88	29.52	36.21
	2DocXL	27.60	13.98	24.46	57.66	76.44	10.54	1.95	9.00	29.67	39.35
Contrastive ICL	Baseline	26.81	13.26	23.86	56.44	72.92	10.42	1.90	8.91	29.41	40.51
	ICL1Doc	29.25	15.82	26.14	56.93	67.41	20.47	11.40	18.96	41.85	33.94
	ICL2Doc	28.62	16.05	25.68	56.07	66.87	23.23	14.66	22.02	43.09	34.20
	ICL1Doc+	**30.62**	**17.45**	**27.79**	**58.96**	**73.86**	25.09	15.87	23.87	**47.12**	**43.50**
	ICL2Doc+	30.24	17.77	27.51	57.55	67.51	**26.01**	**17.46**	**24.90**	47.04	37.24
Focus Mode	Baseline	26.81	13.26	23.86	56.44	72.92	10.42	1.90	8.91	29.41	40.51
	2Doc1S	26.11	12.37	23.05	55.65	73.02	10.77	**2.13**	**9.25**	29.90	41.00
	20Doc20S	28.20	14.48	24.90	58.30	74.02	10.64	1.99	9.11	30.03	39.18
	40Doc40S	28.32	14.54	24.99	**58.36**	**77.95**	10.78	2.02	9.20	30.01	36.20
	80Doc80S	**28.85**	**15.01**	**25.51**	58.33	74.15	10.69	2.04	9.15	29.97	38.09
	120Doc120S	28.36	14.80	25.09	57.99	73.95	10.87	2.09	9.23	30.22	38.88

Conclusion

1. Contrastive In-Context Learning RAG performed best overall, especially on MMLU.
2. Focus Mode RAG ranked second, improving precision by prioritizing relevant sentences.
3. Prompt design remains critical for RAG performance.
4. Knowledge base size is less important than document relevance and quality.

Implications for DocMind

The findings in [4] directly shaped the DocMind retrieval pipeline design. The emphasis on retrieval quality and prompt engineering over knowledge base size validated the approach of using targeted, instructor-curated corpora with carefully constructed prompt templates. Future versions of DocMind may incorporate Focus Mode retrieval and contrastive in-context examples to further improve answer precision.

2.2 MARKET ANALYSIS

The market for artificial intelligence-driven educational technologies has experienced rapid growth in recent years, driven by increasing demand for personalized, scalable, and accessible learning solutions. Universities worldwide are actively exploring AI-based systems to enhance student engagement, improve learning outcomes, and reduce instructional workload. This trend positions DocMind within a highly relevant and expanding market segment.

One of the primary market drivers is the growing expectation of students for on-demand academic support beyond traditional classroom hours. Modern learners increasingly rely on digital platforms to supplement lectures, revise materials, and prepare for assessments. AI-powered academic assistants provide immediate responses and personalized explanations, addressing gaps that traditional Learning Management Systems often fail to cover.

The global Educational Technology (EdTech) market is projected to exceed hundreds of billions of dollars by the end of the decade, with AI-integrated learning platforms expected to become an industry standard rather than an optional enhancement. Institutions are shifting from static content delivery systems toward intelligent, adaptive platforms that can respond to individual learning needs.

DocMind aligns strongly with this market direction by offering a controlled, academic-first AI solution that integrates seamlessly with existing university systems. Unlike general-purpose AI tools, DocMind is specifically designed for higher education environments, ensuring content reliability, institutional control, and compliance with academic standards.

2.3 RELATED PROJECTS AND SIMILAR SYSTEMS

This section surveys existing commercial and research-grade tools that are similar to DocMind in scope, comparing their feature sets, usage limits, and pricing models.

2.3.1 NotebookLM

Positioning and Primary Use Case

NotebookLM is Google's AI-powered research and learning tool as described in [18]. It is designed to help users organize and understand collections of sources — such as Portable Document Format (PDF) files, Google Docs, web pages, and YouTube transcripts — inside notebooks, then interact with those sources through chat and a range of generated learning artifacts.

Google explicitly presents NotebookLM as a tool for students and researchers to generate study guides, summaries, mind maps, flashcards, quizzes, and overviews from their own materials.

Sources, Formats, and Notebook Structure

- Users work inside notebooks. Each notebook contains multiple sources.
- On the free plan, users can have:
  - Up to 100 notebooks
  - Up to 50 sources per notebook
  - Each source up to 500,000 words or 200 MB (no page limit)
- Supported sources include PDF files and Google Docs, web pages and URLs, and YouTube videos and other text-based content.

This structure makes NotebookLM suitable for multi-source projects such as course readings, research literature reviews, or mixed-media study sets.

Core Feature Set

Grounded Chat Over Sources:

- Users interact via chat grounded explicitly in notebook sources.
- Answers cite specific source segments.
- Powered by Gemini 2.5 Flash, enabling improved reasoning and multi-step explanations.

Structured Learning Outputs:

NotebookLM can generate study guides and briefing documents, as well as reports in various formats such as blog-style summaries, glossaries, and character analyses.

Audio Overviews:

- AI podcast-style summaries of source material
- Available in over 50 languages

Video Overviews:

- AI-generated narrated slide decks
- Combines images, quotes, and data from sources

Mind Maps:

- Automatically generated visual representations of concept relationships

Flashcards and Quizzes:

- Auto-generated flashcards and quizzes
- Adjustable difficulty and shareable formats

Integrations and Platform:

- Available on web and mobile (iOS and Android)
- Chat history synchronized across devices
- Integrations with Learning Management System platforms such as Canvas and Schoology

Pricing and Usage Limits

Free Plan:

- Up to 100 notebooks
- 50 sources per notebook
- 500,000 words per source
- 50 chat queries per day
- 3 audio overviews per day

Paid Plan (NotebookLM Plus / Google One AI Premium):

- Approximately 19.99 USD/month
- Up to 500 notebooks
- Up to 300 sources per notebook
- 500 chat queries per day
- 20 audio and 20 video overviews per day

Implication for DocMind:

NotebookLM offers a rich feature set, but free usage is capped at 50 queries per day. Scaling beyond this requires a per-user subscription costing approximately 20 USD/month, making it expensive at institutional scale compared to a centralized system such as DocMind.

2.3.2 Humata

Positioning and Primary Use Case

Humata is marketed as an AI assistant for long and complex documents, especially technical papers and knowledge bases as described in [19]. It enables users to upload documents and ask questions, generate summaries, and compare multiple files with grounded answers.

Supported Content and Interaction Model

- Optimized primarily for PDF documents
- Supports question answering, summarizing, and document comparison
- Includes a document viewer with highlighted answer grounding
- Team and knowledge-based features in higher-tier plans

Core Feature Set

1. Asking questions across all uploaded files
2. Summarization and information extraction
3. Multi-document comparison
4. Unlimited file uploads with page-based usage limits
5. Uses GPT-4-level models in higher tiers

Pricing and Usage Limits

Free Plan:

- Price: 0 USD
- Up to 60 pages total
- Up to 10 answers

Student Plan:

- 1.99 USD/month
- 200 pages/month included
- Additional pages at 0.02 USD per page

Expert Plan:

- 9.99 USD/month
- 500 pages/month included
- Up to 3 users

Implication for DocMind:

Humata's strict page-based pricing makes it unsuitable for large textbooks and multi-course usage at scale, increasing cost and administrative complexity compared to an institutionally deployed DocMind system.

2.3.3 Mindgrasp

Positioning and Primary Use Case

Mindgrasp positions itself as an AI learning and study platform, aimed at transforming educational content into structured study materials as described in [20].

Supported Content Types

- Documents: PDF, DOC/DOCX, PPT/PPTX, TXT
- Media: MP3, MP4, lecture videos, YouTube
- Web links and articles

File Limits:

- PDF files: maximum 250 pages
- DOC/TXT files: maximum 200 MB

Core Feature Set

1. Automatic summaries and smart notes
2. AI-generated flashcards
3. Quiz generation
4. AI tutor for answering questions
5. Unlimited uploads and library storage (with file-level caps)

Pricing and Limits

- No permanent free plan
- 4-day free trial
- Subscription plans range from approximately 5.99 to 10.99 USD/month
- Unlimited questions, summaries, flashcards, and quizzes

Implication for DocMind:

Mindgrasp's subscription-only model and strict per-file page limits make it costly and inconvenient for large-scale academic use. An institutionally deployed solution such as DocMind avoids per-user subscription fees entirely.

2.3.4 AskYourPDF

Positioning and Primary Use Case

AskYourPDF is a document chat and summarization tool focused on interacting with PDF files and multi-document knowledge bases as described in [21].

Supported Content and Features

- Chat with single or multiple documents
- Knowledge base creation
- Summarization and extraction
- Chrome extension, mobile apps, Zotero integration
- API and ChatGPT integration

Model Strategy

AskYourPDF relies on third-party language models including GPT-4o Mini, GPT-4 and GPT-5 variants, Gemini 2.5 Flash and Pro, and Claude Sonnet and Opus. Premium models require additional usage credits even on paid plans.

Pricing and Usage Limits

Free Plan:

- 1 document per day
- 100 pages per document
- 15 MB maximum file size
- 50 questions per day
- GPT-4o Mini only

Paid Plans:

- Premium: approximately 11.99 USD/month
- Pro tiers: approximately 14.99 USD/month
- Up to 6,000 pages per document in higher tiers
- Premium models consume extra credits

Implication for DocMind:

AskYourPDF offers strong integrations but suffers from strict free limits, reliance on third-party models, and unpredictable costs due to credit-based premium model access. A custom system such as DocMind enables predictable pricing and controlled performance at institutional scale.

2.4 MARKET SURVEY SUMMARY

Table 2.3 summarizes the comparative analysis of the reviewed systems against DocMind across key dimensions.

Table 2.3: Market Survey Comparison of Similar Systems

Feature	DocMind	NotebookLM	Humata	Mindgrasp	AskYourPDF
Institutional deployment	Yes	No	No	No	No
Grounded in course docs	Yes	Yes	Yes	Yes	Yes
Role-based access (RBAC)	Yes	No	No	No	No
Free unlimited queries	Config.	50/day	10 total	Trial only	50/day
Arabic language support	Yes	Partial	No	No	Limited
Instructor material mgmt	Yes	No	No	No	No
Admin dashboard	Yes	No	No	No	No
Open infrastructure	Yes	No	No	No	No

The survey confirms that while commercial tools offer polished interfaces and broad feature sets, none are designed for institutional deployment with multi-role access control and instructor-managed knowledge bases. DocMind fills this gap specifically for university environments.

CHAPTER 3

PROPOSED WORK: SYSTEM ANALYSIS AND DESIGN

3.1 SYSTEM OVERVIEW

DocMind is an AI-powered academic learning assistant designed to support university students by enabling interactive, document-grounded learning. The system allows students to ask natural language questions related to their academic materials and receive accurate, context-aware responses that are strictly based on verified documents. By combining conversational artificial intelligence with structured academic content, DocMind enhances the learning experience while maintaining academic integrity.

The system is intended for use in a university-style environment and can be embedded alongside or linked from an institutional portal or learning management system. Unlike generic AI chat tools, DocMind does not rely on unrestricted internet knowledge for retrieval: tutor chat is grounded in instructor-uploaded materials for the selected subject, and document chat is grounded in the student's uploaded PDF files for that conversation. This design approach is consistent with the grounded generation paradigm established in [5].

DocMind supports multiple modes of interaction, including document-based chat and subject-based chat. These modes address different learning scenarios, such as revising lecture notes, clarifying specific topics, or preparing for examinations. The system emphasizes reliability, transparency, and ethical use of artificial intelligence in education.

3.2 SYSTEM OPERATING ENVIRONMENT

The DocMind system is delivered through two client surfaces backed by a single API: a responsive web Single-Page Application (SPA) built with React, Vite, and Tailwind CSS, reachable from desktop and mobile browsers, and a native mobile application for iOS and Android built with Flutter. Both clients consume the same backend HTTP API, so feature parity is maintained at the service layer rather than duplicated per platform.

On the backend, a FastAPI service exposes an HTTP Application Programming Interface (API) under the `/api/...` prefix and coordinates authentication, subject and material management, document and tutor chat, feedback, and administrative operations. Deployments typically use PostgreSQL for relational data together with a vector store for embeddings; the reference implementation supports pgvector or Qdrant, selected by configuration. Authentication is implemented with username and password verification, bcrypt-hashed credentials stored for application users, and signed JSON Web Token (JWT) access tokens with logout via token blocklist, rather than direct Single Sign-On (SSO) with an external university identity provider in the current prototype.

The AI components of the system, including embedding generation and language model inference, may be hosted on dedicated servers or accessed through managed AI services. All system communications occur over secure network protocols to protect user data and institutional resources.

3.3 USER CHARACTERISTICS

DocMind is designed to accommodate a diverse range of users within an academic institution. The primary users are students, who may differ significantly in academic level, learning style, and technical proficiency. Some students may be highly familiar with digital tools, while others may have limited experience with advanced educational technologies. As a result, the system interface is designed to be intuitive, minimalistic, and easy to navigate.

Instructors represent another important user group. They are responsible for uploading course materials in PDF format, within configured size limits, managing subject-specific knowledge bases, and monitoring indexing status for materials attached to their subjects. Instructors are assumed to have basic computer literacy and familiarity with institutional learning platforms.

Administrators form the third user group and are responsible for system monitoring, user management, and policy enforcement. These users typically have higher technical expertise and require access to system logs, usage analytics, and configuration settings. The system design ensures that each user group interacts only with features relevant to their role.

3.4 SYSTEM FUNCTIONS

The DocMind system provides a set of core functions that collectively support academic learning and content management.

One of the primary functions of the system is document ingestion and processing. Documents uploaded by students or instructors are analyzed, segmented into smaller chunks, and converted into numerical vector representations using embedding models. These vectors are stored in a vector database to enable efficient semantic retrieval during user interactions.

Another essential function is conversational interaction. When a user submits a question, the system analyzes the query, retrieves the most relevant document segments, and passes them as context to a language model. The generated response is then presented to the user in a conversational format.

The system persists conversations — including document chat and tutor chat — in the relational database, along with message history, until the user or retention policies delete them. Authenticated API sessions use JWTs with a configurable expiry (for example, twelve hours in the default configuration), after which the user must sign in again; logging out revokes the current token identifier.

3.5 SYSTEM CONSTRAINTS

The DocMind system operates under several constraints that influence its design and behavior. Document chat enforces limits on the number and size of PDF uploads per conversation, which are configurable. The system validates file types and triggers ingestion and vector indexing when files are added. Students may attach additional files to an existing document conversation up to the configured maximum, with background re-indexing keeping retrieval aligned with the corpus.

Another constraint relates to data access. Retrieval does not fetch live web pages or third-party corpora at query time; it searches only the vector indexes built from the relevant uploaded materials. Combined with prompting, this narrows evidence to course documents, while acknowledging that the generative model may still introduce knowledge not present in those files.

Subject-based chat is further constrained by the academic calendar. Each subject belongs to a semester whose lifecycle state (upcoming, active, or archived) is derived from its date window, and students may start new tutor conversations only while that semester is active; subjects in archived or not-yet-started semesters remain readable so that prior conversations can be reviewed, but they reject new chats. Semesters left without dates fail open to active, so content is never silently locked.

Performance constraints also exist, particularly with respect to AI model inference time and resource usage. The system must balance response quality with acceptable response times, especially during periods of high usage.

3.6 ASSUMPTIONS AND DEPENDENCIES

The design of DocMind is based on several assumptions. It is assumed that users have access to stable internet connectivity and compatible devices. It is also assumed that instructors upload accurate and relevant course materials, as the quality of system responses depends directly on the quality of the document corpus.

The system depends on several external components and services, including embedding and text generation APIs. The codebase supports configurable providers such as Google Gemini (the default), OpenAI, and Cohere, selected via environment settings. Additional dependencies include PostgreSQL and a vector index (pgvector or Qdrant). An optional agent layer, disabled by default, can route each user turn through a planner that decides whether to retrieve from the vector store before generation.

Additionally, the system assumes compliance with institutional policies related to data privacy, academic integrity, and ethical AI usage.

3.7 SWOT ANALYSIS

To further evaluate the strategic position of the DocMind system, a Strengths, Weaknesses, Opportunities, and Threats (SWOT) analysis is presented, identifying internal strengths and weaknesses as well as external opportunities and threats.

3.7.1 Strengths

DocMind's primary strength lies in its academic-first design philosophy. The system employs Retrieval-Augmented Generation (RAG) to ensure that all responses are grounded in verified, instructor-approved documents. This approach enhances reliability and minimizes the risk of hallucinating or misleading information.

Another key strength is a clear separation into API, services, and data layers with role-based access control (RBAC) suitable for institutional deployment. Because the React web client and the Flutter mobile client consume this same API, the system reaches students across browsers and native iOS and Android devices without duplicating business logic per platform.

3.7.2 Weaknesses

Despite its advantages, DocMind has several inherent limitations. The reliance on RAG-based architectures may restrict complex reasoning capabilities compared to unrestricted large language models. The quality of system responses is also directly dependent on the quality and completeness of uploaded documents.

Furthermore, the system requires significant AI computation and cloud infrastructure, which may increase operational costs for institutions. The lack of real-time internet access, while intentional for academic integrity, limits the system to pre-uploaded content.

3.7.3 Opportunities

The rapid growth of AI-driven education platforms presents substantial expansion opportunities for DocMind. Increasing demand for continuous academic support, particularly in large universities with high student-to-instructor ratios, creates a strong use case for scalable AI assistants.

DocMind also has the potential to expand across multiple faculties, departments, and institutions, adapting to various academic disciplines.

3.7.4 Threats

DocMind faces competition from general-purpose AI tools that offer broader capabilities and unrestricted knowledge access. While these tools lack academic grounding, their widespread availability may reduce the perceived need for specialized systems.

Additionally, evolving data privacy regulations and ethical AI policies pose potential compliance challenges. Resistance to AI adoption within academic institutions may also impact system acceptance and deployment.

3.8 FUNCTIONAL REQUIREMENTS

Functional requirements specify the core capabilities and behaviors that the DocMind system must support, as defined in accordance with IEEE/ISO/IEC 29148 as described in [9]. These requirements are organized by system functionality and user role.

3.8.1 User Roles and Characteristics

DocMind supports three user roles, each with distinct responsibilities, access privileges, and system expectations.

Student

Students are the primary users of the DocMind system. They interact with the system to ask questions, explore academic content, and clarify concepts related to their enrolled courses. The system must provide an intuitive and accessible interface that does not require prior technical knowledge.

Instructor

Instructors are responsible for managing subject-specific content within the system. They upload verified academic materials in PDF format, manage materials attached to subjects they teach, and review upload and indexing outcomes.

Administrator

Administrators oversee system operation, user management, and policy enforcement. They monitor system usage, manage user roles, and ensure compliance with institutional and ethical guidelines.

3.8.2 Authentication and Authorization Requirements

Table 3.1 presents the authentication and authorization requirements for the DocMind system.

Table 3.1: Authentication and Authorization Requirements

ID	Requirement Description
FR-01	The system shall authenticate users with application credentials and issue signed JWT access tokens; role claims on the token shall drive authorization for students, instructors, and administrators.
FR-02	The system shall support role-based access control for students, instructors, and administrators.
FR-03	The system shall restrict access to features based on the authenticated user role.

3.8.3 Document Management Requirements

Table 3.2 presents the document management requirements for the DocMind system.

Table 3.2: Document Management Requirements

ID	Requirement Description
FR-04	The system shall allow instructors to upload course-related academic documents.
FR-05	The system shall allow students to create document-based chat conversations with one or more PDF uploads, subject to per-file size and per-conversation file-count limits.
FR-06	The system shall allow additional PDF uploads to an existing document conversation within the same limits, and shall process or reject uploads based on type, size, encryption, and indexing rules.
FR-07	The system shall reject duplicate material titles within the same subject (same display name); additional deduplication by file content hash is not implemented in the current codebase.
FR-08	The system shall store document metadata for management and retrieval purposes.

3.8.4 Chat and Interaction Requirements

Table 3.3 presents the chat and interaction requirements for the DocMind system.

Table 3.3: Chat and Interaction Requirements

ID	Requirement Description
FR-09	The system shall allow students to initiate document-based chat sessions.
FR-10	The system shall allow students to initiate subject-based chat sessions using instructor-managed materials.
FR-11	The system shall accept natural language questions from users.
FR-12	The system shall retrieve relevant document segments using semantic similarity techniques.
FR-13	The system shall generate responses grounded in retrieved academic content.
FR-14	The system shall display responses in a conversational format.

3.8.5 Session Management Requirements

Table 3.4 presents the session management requirements for the DocMind system.

Table 3.4: Session Management Requirements

ID	Requirement Description
FR-15	The system shall create a unique persisted conversation for each document-based or tutor-based chat thread, owned by the authenticated user.
FR-16	The system shall enforce a predefined expiration time for JWT access tokens and shall reject requests that use expired or revoked tokens.
FR-17	The system shall support explicit logout by recording revoked token identifiers until they expire.
FR-18	The system shall allow users to delete conversations they own, including associated stored files and vector collections where applicable.

3.8.6 Administrative and Monitoring Requirements

Table 3.5 presents the administrative and monitoring requirements for the DocMind system.

Table 3.5: Administrative and Monitoring Requirements

ID	Requirement Description
FR-19	The system shall provide administrators with access to activity records, aggregated analytics (for example daily message rollups), and chat message feedback associated with subjects.
FR-20	The system shall allow administrators to manage users (including instructors), subjects, semesters, and a global flag that enables or disables student access with a custom message.
FR-21	The system shall expose a health endpoint and structured admin APIs suitable for monitoring integration.
FR-22	The system shall derive each semester's lifecycle state (upcoming, active, or archived) from its optional start and end dates, and shall allow students to start new tutor-chat conversations only on active-semester subjects while keeping past-semester conversations readable.

3.9 NONFUNCTIONAL REQUIREMENTS

Nonfunctional requirements define the quality attributes and constraints under which the DocMind system must operate.

3.9.1 Performance Requirements

- The system shall generate responses within acceptable usability time limits under normal load.
- The system shall support concurrent users without significant performance degradation.
- The system shall optimize document retrieval to minimize latency.

3.9.2 Security Requirements

- All API traffic shall use Transport Layer Security (TLS) in production deployments; passwords shall be stored using strong one-way hashing (bcrypt in the implementation), in line with established web security guidelines as described in [11].
- The system shall prevent unauthorized access to academic content through role checks on every protected route.
- The system shall be designed so that institutional data privacy policies can be applied at deployment time (retention, backups, and optional at-rest database encryption are operator responsibilities), in accordance with applicable data protection regulations as described in [12].

3.9.3 Usability Requirements

- The system shall provide a user-friendly interface suitable for non-technical users.
- The system shall support responsive design for different devices.
- Error messages shall be clear and informative.

3.9.4 Reliability and Availability Requirements

- The system shall maintain high availability during academic terms.
- The system shall recover gracefully from service interruptions.
- The system shall ensure data consistency after failures.

3.9.5 Scalability Requirements

- The system shall support growth in the number of users and documents.
- The system architecture shall allow horizontal scaling of backend services.

3.10 CONSTRAINTS AND LIMITATIONS

The DocMind system is subject to several constraints that limit its operation and scope. Retrieval for answers is limited to embeddings built from the active conversation files in document chat, or subject materials in tutor chat; the product does not crawl the open web for context at query time. The underlying generative model may still reflect parametric knowledge from its training; prompts aim to prioritize retrieved passages, but this is not a formal guarantee of exclusivity.

The system is also constrained by computational resources, particularly those required for AI model inference. These constraints influence decisions related to model selection, caching strategies, and infrastructure deployment.

Additionally, the effectiveness of the system is limited by the quality and completeness of the uploaded academic materials. Incomplete or inaccurate documents may reduce response quality.

3.11 SYSTEM DESIGN

This section presents the detailed technical design of the DocMind system. It translates the requirements defined above into concrete architectural, structural, and interaction-level design decisions.

3.11.1 Design Goals and Principles

The design of the DocMind system is guided by several key principles that ensure reliability, scalability, and academic suitability, consistent with established software engineering practice as described in [8].

- **Modularity**: The system is divided into independent components to simplify development, testing, and maintenance.
- **Scalability**: The design supports an increasing number of users, documents, and interactions without major restructuring.
- **Maintainability**: Clear separation of concerns allows future updates and feature extensions.
- **Security**: Sensitive academic data and user information are protected through access control and encryption.
- **Reliability**: The system ensures consistent and predictable behavior during academic use.

3.11.2 Architectural Overview

DocMind follows a layered, service-oriented architecture. The system architecture consists of the following main layers: the Presentation Layer, the Application Layer, the AI Services Layer, and the Data Storage Layer. Each layer has distinct responsibilities and communicates with adjacent layers through well-defined interfaces.

3.11.3 Presentation Layer Design

The presentation layer is implemented as a React Single-Page Application (SPA) served separately from the API. It provides the user-facing experience for login, student home, document chat, tutor (subject) chat, instructor subject management, and administrator dashboards.

Responsibilities

- User authentication (login form, JWT storage, and protected routes)
- Document upload and subject selection
- Chat interface for question submission, assistant replies, and optional thumbs-up/down feedback on tutor-chat responses
- Error and status message visualization (including student-access gate when disabled by an administrator)

The presentation layer is designed to be lightweight and responsive. It does not perform heavy processing; instead, it delegates all core logic to the backend services.

3.11.4 Application Layer Design

The application layer is implemented in Python as FastAPI routers that depend on service classes and, in turn, repository classes over SQLAlchemy async sessions. It implements business logic, enforces system rules, and manages communication between the presentation layer and AI services.

Core Components

- Authentication and authorization (AuthService, JWT helpers, role dependencies)
- Subject, semester, and material management (SubjectService, SemesterService, MaterialService), where each semester's lifecycle state is derived from its date window at read time rather than stored, so terms roll over automatically as the calendar advances without a scheduler or manual toggle
- Document and tutor chat orchestration (DocumentChatService, TutorChatService) including ingestion hooks
- Administrative services (users, statistics, activity, feedback listing, and system flags)

This layer validates user inputs, enforces constraints such as JWT validity, student-access flags, and upload limits, and ensures that only authorized actions are performed. Administrative mutations record a single structured activity entry per operation for audit-style review.

3.11.5 AI Services Layer Design

The AI services layer is responsible for all intelligent processing within the DocMind system. This layer integrates machine learning models to support semantic understanding and response generation.

Embedding Generation Service

The embedding service converts documents and user queries into numerical vector representations. These embeddings capture semantic meaning and enable similarity-based retrieval.

Key design considerations include chunking large documents into manageable segments, ensuring consistent embedding dimensions, and optimizing embedding generation for performance.

Retrieval Module

The retrieval module searches the vector database to identify document segments most relevant to the user query. Semantic similarity measures are used to rank results.

This module ensures that only the most contextually relevant information is passed to the language model, reducing noise and improving response quality. An optional cross-encoder reranking stage (disabled by default) can over-fetch candidates and re-order them for precision, and the optional agent layer can further restrict retrieval to specific instructor-named materials.

Response Generation Module

The response generation module calls a configurable remote Large Language Model (LLM) — for example Google Gemini, OpenAI, or Cohere Application Programming Interfaces (APIs) — to produce answers from retrieved chunks and prompt templates, with English and Arabic template locales in the codebase. By default, each chat turn follows a single-shot retrieve-then-generate path; when the agent feature flag is enabled, a planner model may decide whether retrieval runs and with which query before generation.

3.11.6 Data Storage Layer Design

The data storage layer manages persistent system data. A hybrid storage strategy is used to balance structured data management and semantic retrieval efficiency.

Relational Database Design

The relational database stores structured metadata, including user accounts and roles, subject information, document metadata, and chat session records. Relational constraints ensure data integrity and consistency across the system.

Vector Database Design

Embeddings and chunk metadata are stored in a pluggable vector backend: either PostgreSQL with the pgvector extension as described in [16] (sharing the primary application database URL) or a Qdrant collection as described in [17] (file- or server-backed, depending on deployment). Each embedding is associated with conversation or material identifiers so that retrieval scopes to the correct corpus.

This design enables semantic search while keeping the relational model as the source of truth for ownership and permissions.

3.11.7 System Block Diagram

Figure 3.1 presents the system block diagram, representing a multi-channel application architecture that supports both web and mobile clients, backed by a centralized API, authentication services, and a Retrieval-Augmented Generation (RAG) component.

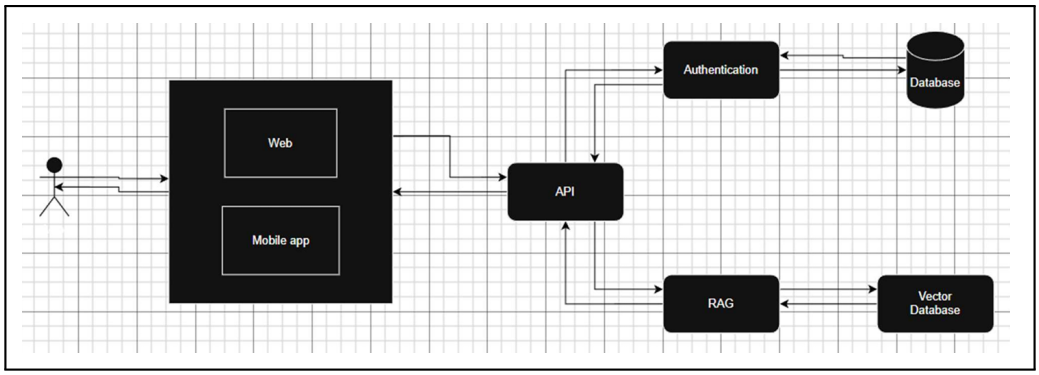


Figure 3.1: DocMind System Block Diagram.

3.11.8 Use Case Diagram

Figure 3.2 presents the use case diagram, which illustrates the interactions between system actors and core functionalities. It defines how students, instructors, and administrators interact with DocMind at a high level.

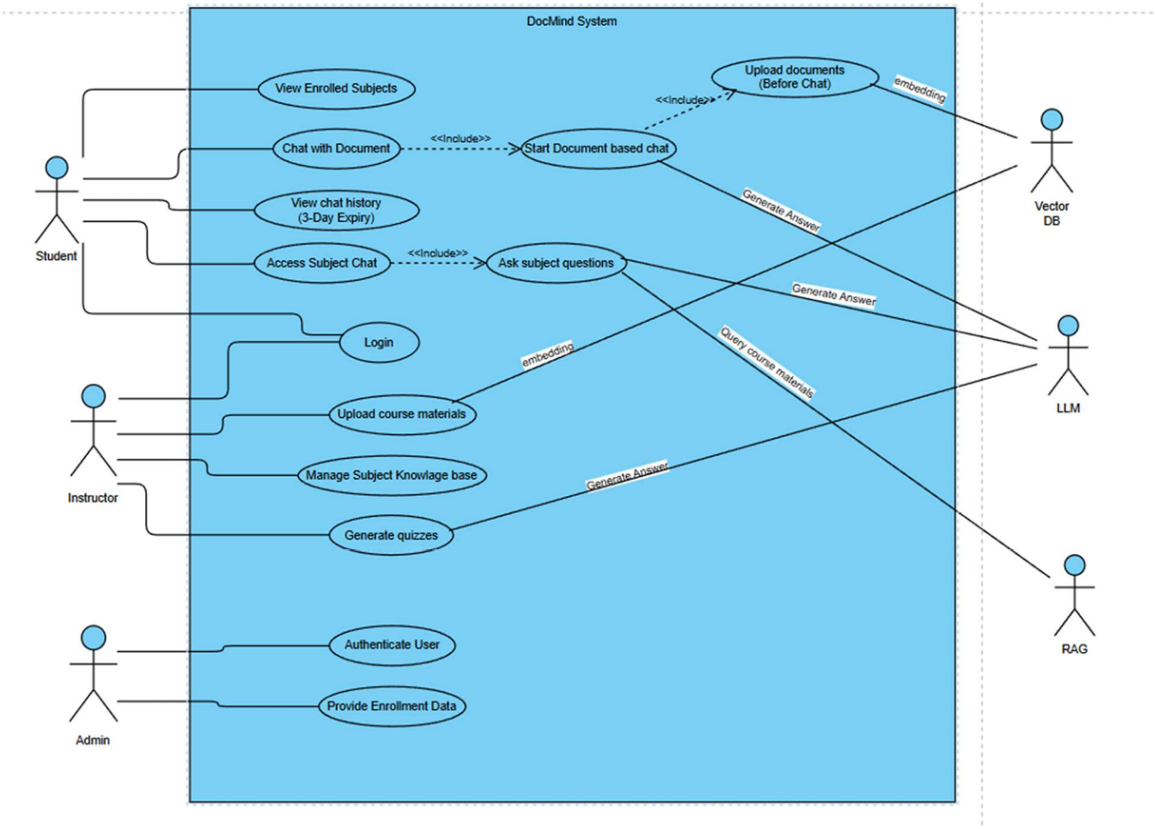


Figure 3.2: DocMind Use Case Diagram.

3.11.9 Activity Diagram

Figure 3.3 presents the activity diagram, which describes the flow of actions within the system, particularly for document-based chat interactions from the student's perspective. These diagrams clarify decision points, parallel actions, and system responses throughout the interaction process.

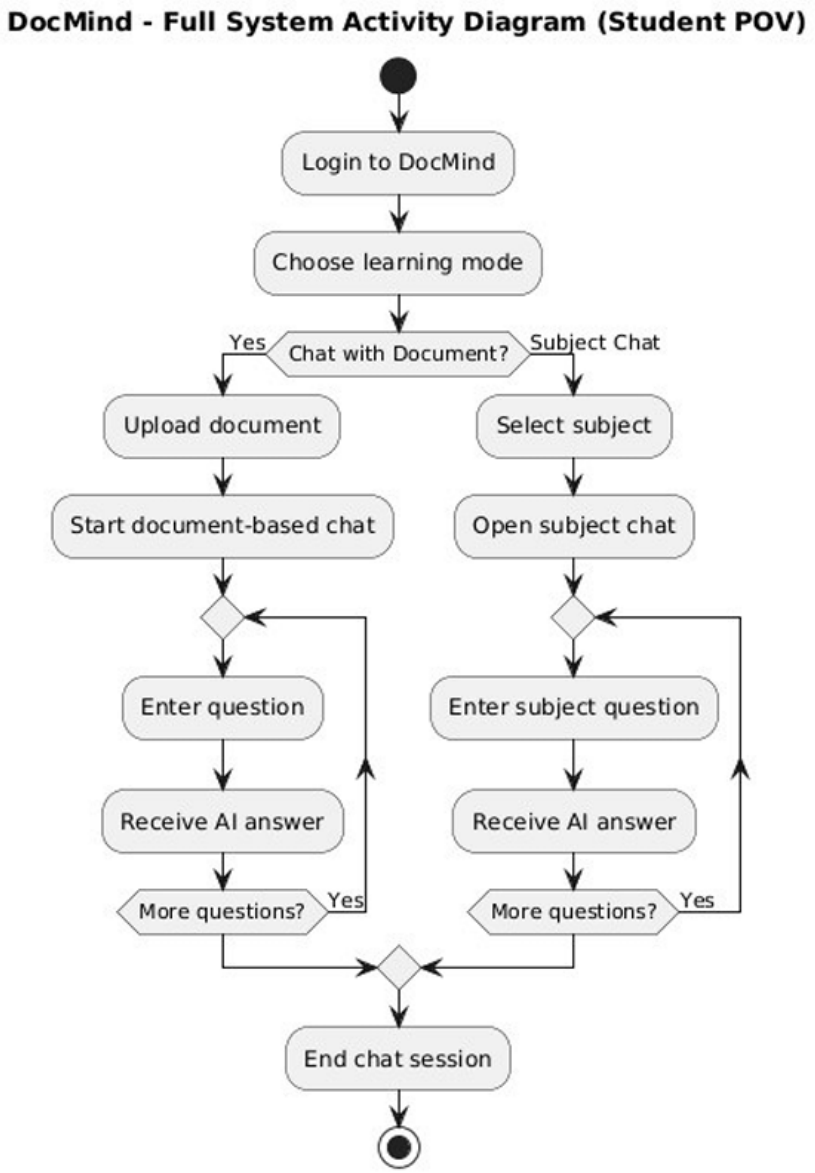


Figure 3.3: DocMind Full System Activity Diagram (Student POV).

3.11.10 Sequence Diagrams

Sequence diagrams illustrate the time-ordered interaction between system components for the two primary chat flows.

Document-Based Chat (RAG)

Figure 3.4 presents the sequence diagram for the document-based chat flow, showing how file upload and vector checking are orchestrated between the frontend, backend, and vector store.

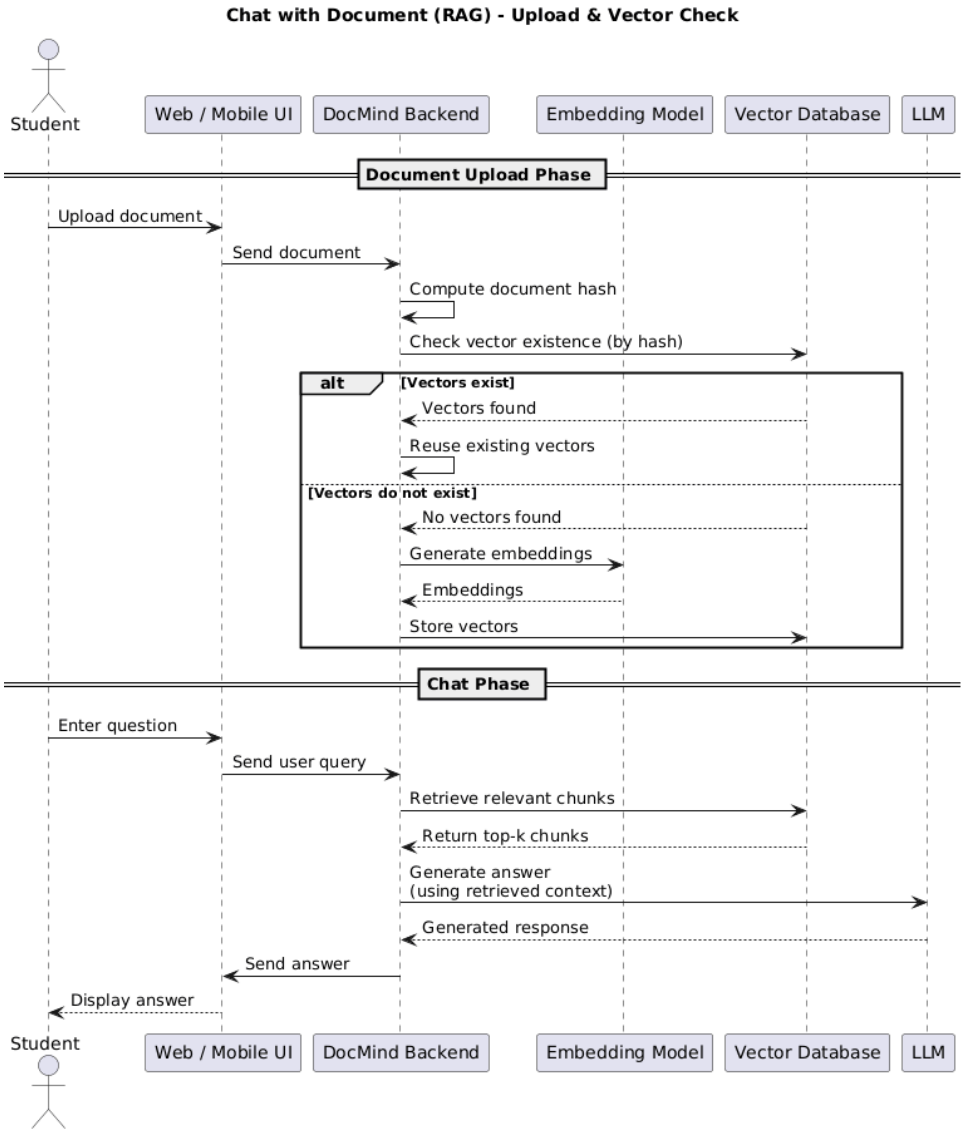


Figure 3.4: Chat with Document (RAG) – Upload and Vector Check Sequence Diagram.

Subject-Based Chat

Figure 3.5 presents the sequence diagram for the subject-based chat flow, illustrating how instructor-uploaded materials are retrieved and used to ground the AI response.

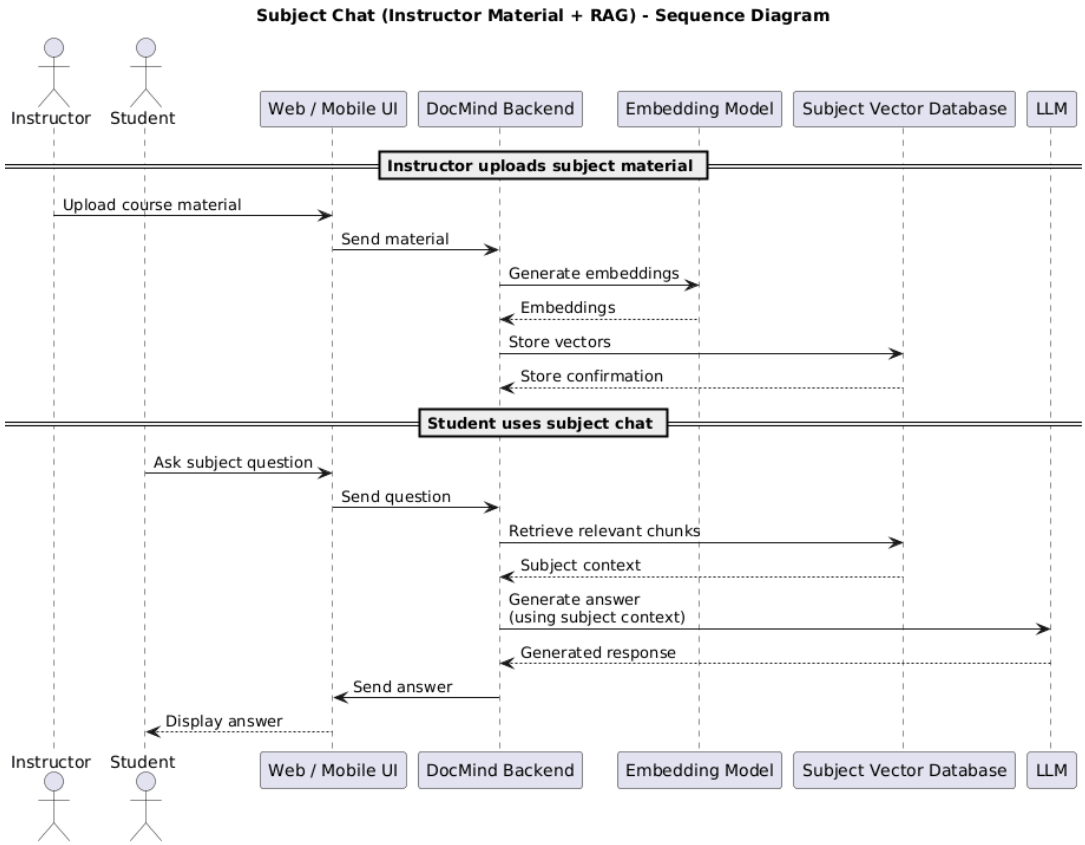


Figure 3.5: Subject Chat (Instructor Material and RAG) – Sequence Diagram.

3.11.11 Database Design and ERD

The database design is represented using an Entity-Relationship Diagram (ERD), which illustrates entities, attributes, and relationships. Figure 3.6 presents the full ERD for the DocMind system.

Entity-Level Design

Each entity in the ERD shown in Figure 3.6 corresponds to a logical system component. Primary and foreign keys enforce relationships between users, subjects, documents, sessions, and embeddings. Subjects optionally reference a semester through a nullable foreign key that is cleared rather than cascaded when a semester is removed; each semester stores an optional start and end date from which its lifecycle state is derived in application code, so the state itself is not persisted.

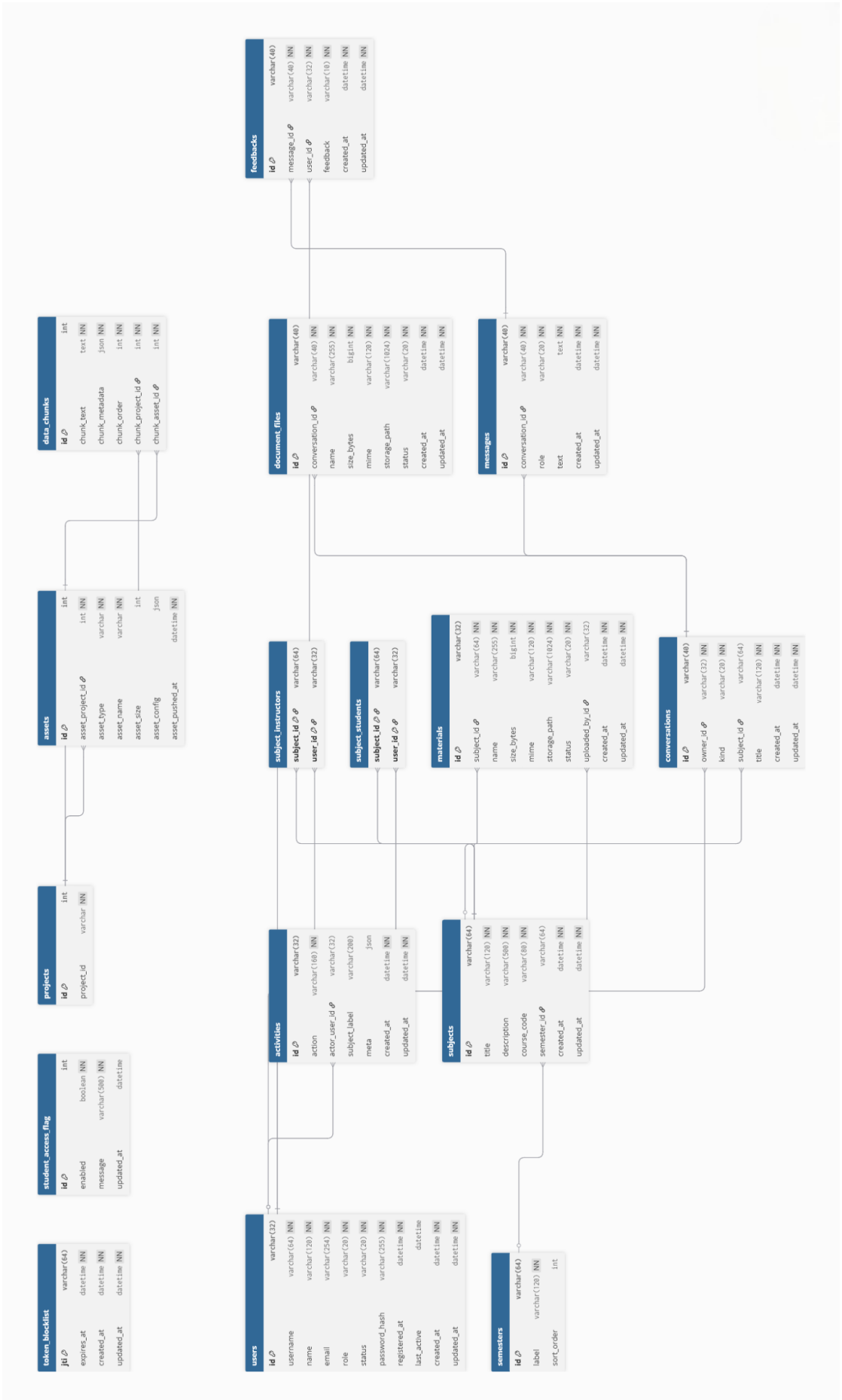


Figure 3.6: DocMind Entity-Relationship Diagram (ERD).

3.11.12 Security Design

Security is integrated into the system design at multiple levels. Authentication is handled within the application using password verification and JWT issuance; authorization is enforced through role-based access control on each route.

Production deployments are expected to use HTTPS end-to-end; passwords are stored hashed. Structured activity and admin endpoints support operational auditing; deeper anomaly detection would be added at the infrastructure layer.

3.11.13 Error Handling and Fault Tolerance

The system is designed to handle errors gracefully. Invalid inputs, expired sessions, unavailable services, and unexpected failures are managed through controlled error responses.

Fallback mechanisms and retries are implemented where appropriate to improve system robustness.

3.11.14 Design Validation

The design decisions described in this chapter are validated against the requirements defined in the previous sections. Each functional requirement is supported by one or more design components, ensuring full traceability.

This structured system design provides a solid foundation for implementation, testing, and future system enhancement.

CHAPTER 4

IMPLEMENTATION AND TESTING

This chapter describes the working implementation in the DocMind repository: a full-stack prototype comprising a FastAPI backend, PostgreSQL schema with Alembic migrations, optional pgvector or Qdrant for embeddings, and a React frontend. It demonstrates how the concepts, requirements, and design decisions from Chapter 3 are realized in code and in the running application.

The implementation covers the primary user journeys for students, instructors, and administrators, including authentication, document chat with file lifecycle and feedback, tutor (subject) chat over indexed materials, instructor material upload, and admin dashboards for users, subjects, analytics, feedback, activity, and student-access control.

4.1 TECHNOLOGY STACK

The DocMind system is implemented using the following technologies:

- **Backend**: Python 3.10, FastAPI as described in [13], SQLAlchemy (async), Alembic for migrations, Pydantic for data validation.
- **AI/ML**: an in-house Retrieval-Augmented Generation (RAG) orchestration layer (structure-aware chunking, prompt assembly, and retrieval) that uses LangChain text-splitting utilities as described in [14]; embedding generation and text generation are served by configurable remote providers — Google Gemini by default, switchable to OpenAI or Cohere — and an optional `sentence-transformers` cross-encoder is available for reranking.
- **Vector Store**: pgvector as described in [16] (PostgreSQL extension) or Qdrant as described in [17], switchable by environment configuration.
- **Relational Database**: PostgreSQL 16 with the pgvector extension.
- **Web Frontend**: React 19 with Vite, Tailwind CSS, React Router, and Axios for API calls.

- **Mobile**: Flutter and Dart, with GetX for state management and dependency injection and Dio for HTTP, organized in Clean Architecture (data, domain, and presentation layers per feature).
- **Authentication**: JSON Web Tokens (JWT), bcrypt password hashing, and token blocklist for logout.
- **Containerization**: Docker and Docker Compose for local development and deployment.
- **Version Control**: Git with GitHub, using a monorepo structure with separate backend, frontend, and mobile directories.

4.2 BACKEND IMPLEMENTATION

4.2.1 Project Structure

The backend follows a layered architecture:

- **Routers** (`/api/...`): Handle HTTP request routing and response serialization.
- **Services**: Contain business logic, orchestrate RAG operations, and enforce authorization constraints.
- **Repositories**: Abstract database access using SQLAlchemy async sessions.
- **Models**: Define SQLAlchemy ORM models and Pydantic schemas.
- **Core**: Configuration management (`settings.py`), security utilities, and dependency injection.

4.2.2 Authentication and Authorization

JSON Web Token (JWT)-based authentication is implemented in AuthService. On successful login, the service issues a signed access token embedding the user's identifier and role. Every protected route uses a FastAPI dependency that validates the token signature, checks expiry, and rejects token identifiers recorded in the blocklist. Role-specific dependencies — `require_student`, `require_instructor`, and `require_admin` — are applied to route groups to enforce the principle of least privilege.

4.2.3 Document Ingestion Pipeline

When a student uploads one or more PDF files to a document-based conversation, the backend performs the following steps:

1. Validates the file type, MIME type, size, and per-conversation count against configured limits.
2. Stores the file on disk and records metadata in the `documents` table.
3. Triggers an asynchronous ingestion task that reads the PDF and splits it into structure-aware chunks: detected tables are emitted as atomic Markdown blocks, section headings (detected by relative font size) are prepended to each chunk as breadcrumbs, and prose is split recursively (paragraph, then line, then sentence) with a trailing-text overlap so that definitions and worked examples are not cut mid-thought. It then generates embeddings for each chunk and upserts the vectors into the conversation-scoped collection in the vector store.
4. Updates the document's indexing status in the relational database so the frontend can display the current state (pending, indexed, or failed).

4.2.4 RAG Query Pipeline

When a student submits a message, the backend performs the following steps:

1. The backend retrieves the top-k most semantically similar chunks from the conversation or subject collection. When cross-encoder reranking is enabled (disabled by default), it over-fetches a larger candidate set by recall and a cross-encoder truncates it back to the top-k by precision, soft-degrading to the original vector order on any fault.
2. Each retrieved chunk carries its provenance (source filename, section heading, and page or slide); these are assembled into a context block, inserted into a prompt template available in English and Arabic locales, and the model is instructed to cite the sources it relied on. For subject chat, a short manifest of the subject's indexed materials is also supplied so the model knows what it can ground answers in.
3. The prompt is sent to the configured LLM API (Gemini, OpenAI, or Cohere) for generation.
4. The response is stored in the `messages` table, associated with the conversation, and returned to the client.
5. Optionally, if the agentic mode is enabled, a planner model first decides whether retrieval is needed and with what reformulated query before the generator runs; it may additionally scope retrieval to specific instructor materials the student names (disabled by default).

4.2.5 Administrative APIs

The admin router exposes endpoints for user Create, Read, Update, and Delete (CRUD) operations (create instructor accounts, activate or deactivate users, reset passwords), subject management, semester management (creating, updating, and deleting semesters with optional date windows, where subjects of a deleted semester are left unassigned rather than removed), aggregated statistics (daily message counts, subject-level activity, per-user usage), feedback records collected from student thumbs-up/down interactions, a global student-access flag that gates all student-facing routes with a configurable maintenance message, and a structured activity log recording every administrative action for audit review.

4.3 FRONTEND IMPLEMENTATION

4.3.1 Application Structure

The React frontend is structured as a Single-Page Application (SPA) with protected routes guarded by the stored JWT. Navigation is organized around three role-specific views:

- **Student**: Home dashboard, document-based chat, subject selection, and tutor chat.
- **Instructor**: Subject management dashboard with material upload, status tracking, and file management.
- **Administrator**: User management, subject management, semester management, analytics, feedback viewer, activity log, and system settings.

4.3.2 State Management

Application state is managed via React Context for authentication (token storage, decoded role, and logout trigger) and component-level state for chat history, upload progress, and indexing status. The frontend polls or listens for indexing status updates so that students receive real-time feedback during document processing.

4.4 STUDENT INTERFACE

4.4.1 Document-Based Chat Interface

In document-based chat, students create a conversation with one or more PDF uploads within configured count and size limits. The backend saves files, runs ingestion and chunk embedding, and attaches vectors to a per-conversation collection. Students may add further PDF files to the same conversation up to the limit, then ask questions in a conversational interface while indexing states are shown in the User Interface (UI).

The chat interface displays user questions and system responses in a chronological format, allowing students to follow the conversation flow easily. Visual indicators are used to show system processing states, such as document indexing and response generation.

Figure 4.1 shows the student interface for document-based chat.

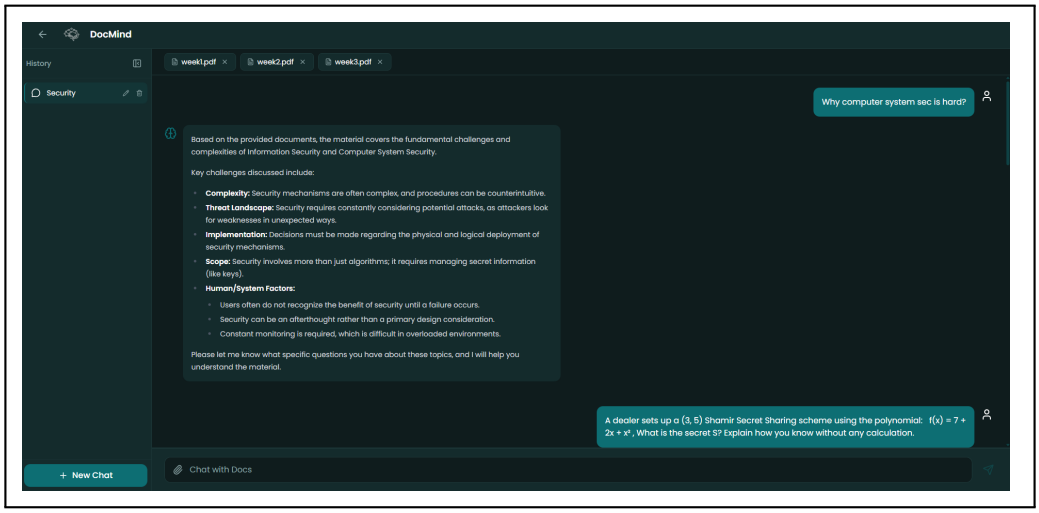


Figure 4.1: Student Interface – Document-Based Chat.

4.4.2 Subject-Based Chat Interface

The tutor (subject-based) chat interface allows students to select a course or subject for which they are enrolled and for which instructors have uploaded materials. Subjects are presented with the most recent semester first, so the current term surfaces above older ones. Retrieval is limited to indexed content for that subject. The interface surfaces the active subject and instructor context so that answers remain visually tied to the chosen course. Subjects belonging to an archived semester are shown in a read-only state: their past conversations remain reviewable, but starting a new chat is disabled.

Figure 4.2 shows the student interface for subject-based chat.

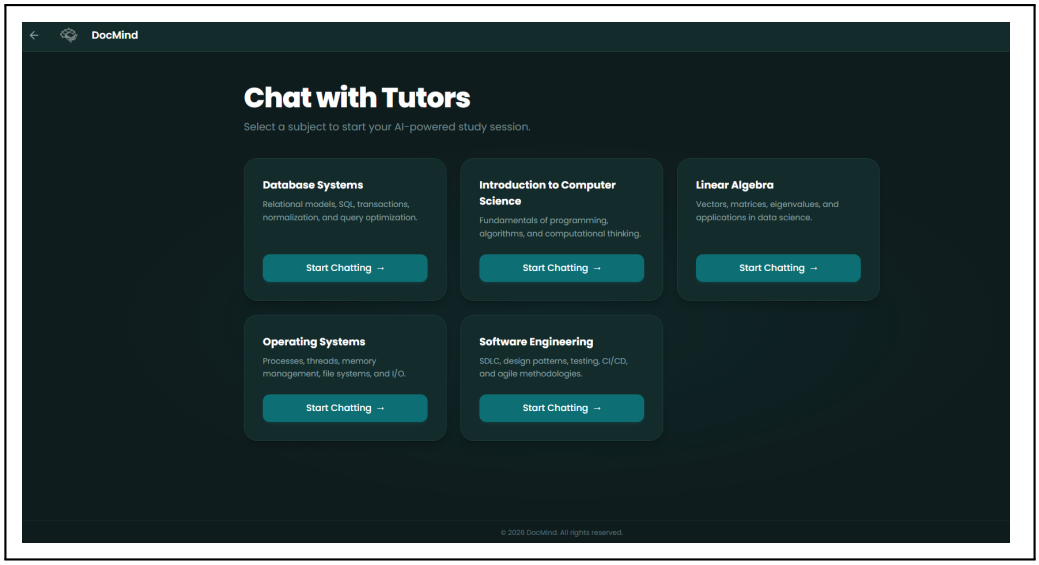


Figure 4.2: Student Interface – Subject-Based Chat (Chat with Tutors).

4.5 INSTRUCTOR INTERFACE

The instructor interface focuses on content management rather than acting as the chat tutor in the product flow. Instructors use a dashboard to open each assigned subject, upload or replace PDF materials, and observe processing status; aggregate analytics and cross-subject admin views are reserved for the administrator role in the current implementation.

4.5.1 Content Management Interface

Instructors can upload lecture notes, slides, and reference documents through a structured upload interface. The system automatically processes uploaded content and integrates it into the subject-specific knowledge base.

The interface provides feedback on upload status; the API rejects a new material when another material with the same display name already exists for that subject, which reduces accidental duplicate titles, though file-level byte deduplication is not implemented in the current codebase.

Figure 4.3 shows the instructor content management dashboard.

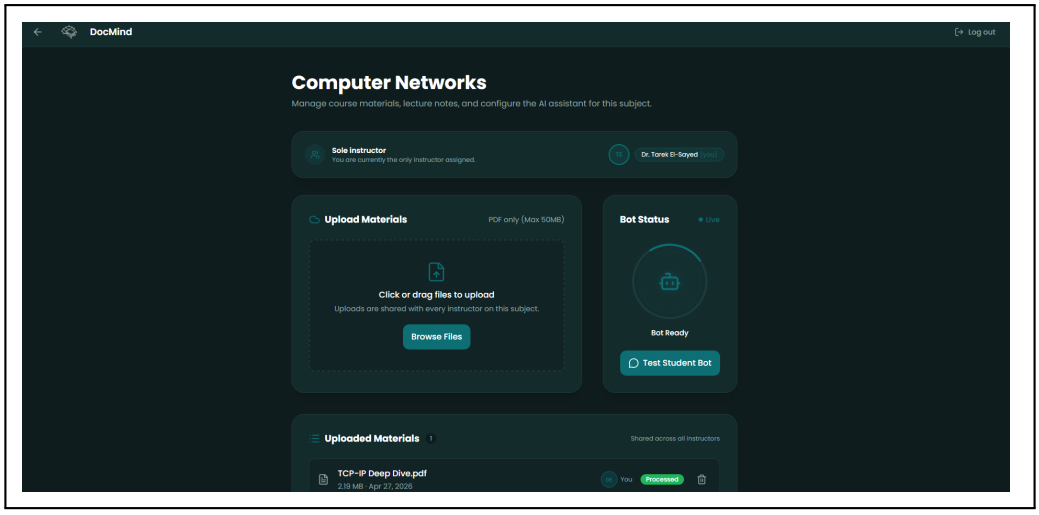


Figure 4.3: Instructor Interface – Content Management Dashboard.

4.6 TESTING

4.6.1 Testing Strategy

The DocMind system employs a multi-level testing strategy to validate correctness, security, and usability:

- **Unit Testing**: Individual service functions and repository methods are tested in isolation using pytest with async support. Mock objects are used to substitute external dependencies such as the vector store and LLM API.
- **Integration Testing**: API endpoints are tested end-to-end using an asynchronous HTTP client (`httpx.AsyncClient`) that drives the application in-process, exercising the full request-response cycle — including authentication, authorization, and database interaction — against a dedicated PostgreSQL with pgvector test database, while the language model and vector store are substituted with test doubles.
- **RAG Pipeline Testing**: The embedding and retrieval pipeline is validated by ingesting known documents and verifying that expected chunks are retrieved for representative queries.
- **Security Testing**: Role-Based Access Control (RBAC) is validated by asserting that requests made with student-role tokens are rejected by instructor and administrator endpoints, and vice versa.
- **User Acceptance Testing (UAT)**: Manual walkthroughs of the primary student, instructor, and administrator user journeys are performed against the running application with seeded data.

4.6.2 Test Environment

Testing is performed against a local Docker Compose environment that mirrors the production stack. Seeded users and subjects provide demo accounts and sample materials for repeatable test scenarios.

4.6.3 Test Cases

Table 4.1 summarizes representative test cases executed during system validation.

Table 4.1: Representative System Test Cases

Test ID	Description	Expected Result	Status
TC-01	Student login with valid credentials	JWT issued, dashboard loaded	Pass
TC-02	Student login with invalid password	401 Unauthorized returned	Pass
TC-03	Student accesses admin endpoint	403 Forbidden returned	Pass
TC-04	Upload valid PDF to document chat	File ingested, indexed, status shown	Pass
TC-05	Upload invalid file type (DOCX)	422 Unprocessable Entity returned	Pass
TC-06	Ask question in document chat	RAG pipeline runs, answer returned	Pass
TC-07	Ask question in subject chat	Subject-scoped retrieval, answer returned	Pass
TC-08	Instructor uploads material with duplicate title	409 Conflict returned	Pass
TC-09	Admin disables student access	Students see maintenance message	Pass
TC-10	Admin views analytics dashboard	Aggregated stats displayed correctly	Pass
TC-11	Student submits thumbs-up feedback	Feedback stored, retrievable by admin	Pass
TC-12	Logout and reuse token	401 Unauthorized (token blocklisted)	Pass
TC-13	Student starts tutor chat in an archived-semester subject	403 Forbidden; past conversations remain readable	Pass

4.6.4 Known Limitations and Remaining Gaps

Remaining gaps are mainly operational and product-level rather than missing core flows. There is no native Single Sign-On (SSO) integration with a campus identity provider yet; encryption at rest and formal retention policies depend on hosting choices; rate limiting and horizontal autoscaling are not described in code as fixed guarantees; and automated test coverage is still growing.

Language models may still produce incorrect or out-of-context text despite RAG; the implementation mitigates this through retrieval, prompts, and optional agent routing, but does not eliminate model risk entirely.

4.7 PROTOTYPE EVALUATION

Initial evaluation of the running system indicates that document-grounded and subject-grounded conversational learning paths function end-to-end with the stack described in this document. Stakeholder feedback and usage metrics, including optional per-message feedback stored for administrators, should inform further refinements to prompts, retrieval parameters, and User Experience (UX) design. This chapter documents the transition from system design to a concrete repository-backed implementation that can be demonstrated and extended toward production hardening.

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

5.1 CONCLUSIONS

The DocMind project successfully demonstrates the feasibility of integrating a trustworthy, document-grounded AI assistant into a university academic environment. By employing Retrieval-Augmented Generation (RAG) over explicitly managed document corpora, the system substantially reduces the risk of hallucination compared to unrestricted language model deployments, ensuring that student responses remain anchored to verified course materials.

The implemented system achieves all primary objectives set out in Chapter 1. Role-Based Access Control (RBAC) effectively separates student, instructor, and administrator concerns. The dual interaction model — encompassing both document-based chat and subject-based tutor chat — addresses different student learning scenarios within a unified interface. JSON Web Token (JWT)-based authentication with configurable token lifetime and a blocklist-based logout mechanism provides session security appropriate for institutional deployment.

The literature review confirmed that plain RAG, without necessarily fine-tuning the underlying model, is a practical and cost-effective strategy for domain-specific question-and-answer applications as established in [5]. Paper findings from Barnett et al. as described in [3] reinforced this by demonstrating that fine-tuning may degrade RAG performance in specialized domains, while Li et al. as described in [4] showed that prompt engineering and retrieval quality have greater impact on answer accuracy than knowledge base size. These insights directly shaped the DocMind retrieval pipeline design.

The market survey revealed that while commercial tools such as NotebookLM as described in [18], Humata as described in [19], Mindgrasp as described in [20], and AskYourPDF as described in [21] offer overlapping document-chat features, none are designed for institutional multi-role deployment with instructor-controlled knowledge bases. DocMind fills this gap by providing an open, configurable system that institutions can deploy and operate without per-user subscription costs.

In conclusion, DocMind demonstrates that it is possible to build an academically responsible AI assistant that is reliable, role-aware, and deployable within university infrastructure using accessible open-source technologies. The system represents a meaningful contribution to the intersection of artificial intelligence and higher education, providing a foundation that can be extended, hardened for production, and adopted by institutions seeking to improve student learning outcomes without compromising academic integrity.

5.2 FUTURE WORK

While the current implementation realizes the core functionality of DocMind, several enhancements are planned to increase the system's robustness, reach, and academic utility. The following subsections describe the most significant planned directions.

5.2.1 Single Sign-On Integration

The current authentication model uses application-managed credentials. Future work will extend the authentication layer to support Security Assertion Markup Language (SAML) 2.0 or OAuth 2.0 federation with institutional identity providers (such as Lightweight Directory Access Protocol (LDAP), Active Directory, or university Single Sign-On (SSO) portals), eliminating the need for separate account provisioning and aligning with institutional security policies.

5.2.2 Multi-Language Support

Although the system includes English and Arabic prompt templates, the user interface is primarily English. Future versions will provide a fully localized Arabic interface and extend the language model prompt templates to support additional languages relevant to the institution's student population.

5.2.3 Automated Quiz and Assessment Generation

A natural extension of the subject-based chat is the ability for instructors to trigger AI-generated quiz questions from uploaded materials. These would be reviewed and published by the instructor before being made available to students, maintaining instructor control while automating a time-consuming task.

5.2.4 Enhanced Retrieval Strategies

The RAG pipeline already supports optional cross-encoder reranking to improve the precision of the top-k retrieved chunks, and optional source-scoped retrieval that restricts a query to specific instructor materials; both are disabled by default and can be enabled per deployment. Future improvements, informed by best-practice findings in [4], include hybrid retrieval combining dense vector search with keyword-based (BM25) sparse retrieval, and query expansion using lightweight models to improve recall for short or ambiguous student queries.

5.2.5 Analytics and Learning Insights

The administrator analytics dashboard currently provides usage-level metrics. Future work will introduce learning-oriented analytics, such as frequently asked concept clusters per subject, knowledge gap detection based on question patterns, and per-student engagement trends. These insights would assist instructors in adapting their course materials to student needs.

5.2.6 Production Hardening

Operational improvements include formal rate limiting on all API endpoints to prevent abuse, encryption at rest for the relational database and vector store, automated integration and regression test coverage tracking, horizontal autoscaling configuration for cloud deployments using Kubernetes-based orchestration, and formal data retention and deletion policies compliant with the General Data Protection Regulation (GDPR) as described in [12] and institutional data governance frameworks.

5.2.7 Agentic Retrieval Expansion

The optional agentic layer (plan-execute-reflect) is disabled by default in the current implementation. Future work will mature this component, evaluate its impact on answer quality and latency, and promote it as the default retrieval strategy once stability is confirmed.

REFERENCES

[1] H. Zamani and A. Salemi, "Comparing Retrieval-Augmentation and Parameter Efficient Fine-Tuning for Privacy-Preserving Personalization of Large Language Models," University of Massachusetts, Jun. 2025.

[2] N. Lawton, A. Samuel, A. Kumar, and D. Liu, "A Comparison of Independent and Joint Fine-Tuning Strategies for Retrieval-Augmented Generation," Cornell University, Oct. 2025.

[3] S. Barnett, Z. Brannelly, Stefanus, and S. Wong, "Fine-Tuning or Fine-Failing? Debunking Performance Myths in Large Language Models," Deakin University, Jun. 2025.

[4] S. Li, L. Stenzel, C. Eickhoff, and S. Ali, "Enhancing Retrieval-Augmented Generation: A Study of Best Practices," University of Tübingen, Jan. 2025.

[5] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[6] W. Holmes, M. Bialik, and C. Fadel, *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Boston, MA: Center for Curriculum Redesign, 2019.

[7] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Upper Saddle River, NJ: Pearson, 2023.

[8] I. Sommerville, *Software Engineering*, 10th ed. Harlow, UK: Pearson, 2016.

[9] ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, IEEE Standards Association, 2018.

[10] IEEE Std 1016-2009, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*, IEEE Computer Society, 2009.

[11] OWASP Foundation, *OWASP Top Ten Web Application Security Risks*, 2023. [Online]. Available: https://owasp.org/www-project-top-ten/

[12] European Union, "General Data Protection Regulation (GDPR)," Regulation (EU) 2016/679, Off. J. Eur. Union, Apr. 2016.

[13] S. Tiangolo, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. [Online]. Available: https://fastapi.tiangolo.com

[14] J. Harrison, *LangChain: Building Applications with LLMs*. [Online]. Available: https://python.langchain.com

[15] A. Johnson et al., "FAISS: A Library for Efficient Similarity Search," Facebook AI Research. [Online]. Available: https://github.com/facebookresearch/faiss

[16] pgvector Contributors, *pgvector: Open-Source Vector Similarity Search for PostgreSQL*. [Online]. Available: https://github.com/pgvector/pgvector

[17] Qdrant Team, *Qdrant Vector Database Documentation*. [Online]. Available: https://qdrant.tech/documentation/

[18] Google, *NotebookLM: AI-Powered Research and Learning Tool*. [Online]. Available: https://notebooklm.google.com

[19] Humata AI, *Humata: AI-Powered Document Assistant*. [Online]. Available: https://www.humata.ai

[20] Mindgrasp, *Mindgrasp: AI Learning and Study Platform*. [Online]. Available: https://mindgrasp.ai

[21] AskYourPDF, *AskYourPDF: Chat with Your Documents*. [Online]. Available: https://askyourpdf.com